Credit card data--Network Analysis

Emphasis on detection of community of fraudsters

Last amended: 18th Oct, 2026
My folder: D:\data\OneDrive\Documents\gephi\credit_card_data
GitHub repo: Hadoop AND this pdf file.
Ref for filter: https://seinecle.github.io/gephi-tutorials/generated-html/using-filters-en.html
gdrive folder: credit_card_tranactions
Data source: https://ibm.ent.box.com/v/tabformer-data/folder/130747715605#

Table of Contents

Objectives: 3
Warning: 3
Modules import 3
Data processing 3
Data 3
Top rows: 4
Bottom rows: 4
Proportion of fraud to no-fraud: 4
User Nodes: 5
Merchant Nodes: 6
Edges 8
Back to gephi 8
Import User nodes 8
Import Merchant nodes 11
Import edges 14
Graph processing: 15
III Differentiating Users and Merchants nodes: 17
IV: Differentiating 'Is Fraud' vs no-fraud nodes 18
V: Projecting two mode graphs to one mode 19
Bimodal Projection 20
Analysing network with filters and communities: 21
Plan 21
Modularity 23
Attributes filter? 25
Masking operator 33
Edge Weight filter: 35
Interpretation of Filtered Graph 36
Prompt: 36

Reply: 36
Prompt: 37
Reply 37

Work in Windows rather than in VM.. It is fast and convenient.

Objectives:

To discover community of users who collaborate to commit fraud. In the process, we learn:

- Learn to process tabular data for import into gephi
- Learn to work with Multimode data
- Learn to work with filters in gephi

Warning:

Gephi consumes a lot of memory. Perform this experiment by directly installing gephi in Windows. Then, give gephi more memory to work with by making changes to its file: "*D:\Program Files\Gephi-0.9.2\etc\gephi.conf*". See figure below:

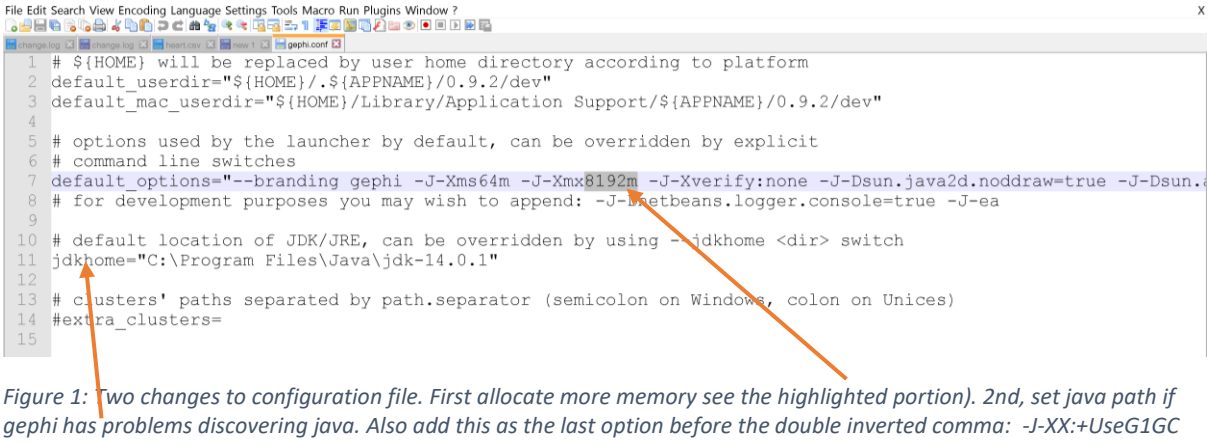


Figure 1: Two changes to configuration file. First allocate more memory see the highlighted portion). 2nd, set java path if gephi has problems discovering java. Also add this as the last option before the double inverted comma: -J-XX:+UseG1GC

Also add this as the last option before the double inverted comma: -J-XX:+UseG1GC

Modules import

After installing gephi, please click on *Tools→Plugins* and import the following modules. For this experiment we need, just the first one.

1. MultimodeNetworksTransformationPlugin
2. Newman-Girvan Clustering
3. Leiden Algorithm
4. Map of countries
5. MultiGravity ForceAtlas 2
6. GeoLayout
7. Clustering Coefficient

Data processing

Data

Data file is more than 2gb. We only extract that information from it that we need. And then carry out some pre-processing on the extracted data so as to make them fit for import into gephi. See this google colab file for data pre-processing under '*hadoop'* repo in github.

Actual data is very large. It is from year 2002 to 2020. Number of users are 2000.While with predictive analysis, we may be able to predict pattern of frauds, we want to find out if certain users are acting in collusion. Here are our data, processing steps and sample data:

```
# 3.1 Our data:
df.head()
df.tail()
df.shape   # (24386900, 6)
```

Figure 2: Around 2.5 crore observations

Top rows:

	User	Year	Amount	Merchant Name	Merchant City	MCC	Is Fraud?
0	0	2002	$134.09	3527213246127876953	La Verne	5300	No
1	0	2002	$38.48	-727612092139916043	Monterey Park	5411	No
2	0	2002	$120.34	-727612092139916043	Monterey Park	5411	No
3	0	2002	$128.95	3414527459579106770	Monterey Park	5651	No
4	0	2002	$104.71	5817218446178736267	La Verne	5912	No

Figure 3: Our data

Bottom rows:

]:

	User	Year	Amount	Merchant Name	Merchant City	MCC	Is Fraud?
24386895	1999	2020	$-54.00	-5162038175624867091	Merrimack	5541	No
24386896	1999	2020	$54.00	-5162038175624867091	Merrimack	5541	No
24386897	1999	2020	$59.15	2500998799892805156	Merrimack	4121	No
24386898	1999	2020	$43.12	2500998799892805156	Merrimack	4121	No
24386899	1999	2020	$45.13	4751695835751691036	Merrimack	5814	No

Proportion of fraud to no-fraud:

	proportion
Is Fraud?	
No	0.99878
Yes	0.00122

We will take a sample by considering data only for the year 2002. Our sample size is 350732.

```
# 4.1 Sample size:
dfsample.shape   # (350732, 6)
```

```
(350732, 7)
```

```
# 4.2 How many incidents of frauds?
dfsample['Is Fraud?'].value_counts()
```

Is Fraud?	count
No	350593
Yes	139

Sample data. We rewrite usernames as u0, u1,..and also shorten merchant names. This is our data:

	User	Year	Amount	Merchant Name	Merchant City	MCC	Is Fraud?
0	u0	2002	134.09	m7466	La Verne	5300	No
1	u0	2002	38.48	m4998	Monterey Park	5411	No
2	u0	2002	120.34	m4998	Monterey Park	5411	No
3	u0	2002	128.95	m7393	Monterey Park	5651	No
4	u0	2002	104.71	m8834	La Verne	5912	No

Figure 4: Sample data

User Nodes:

Every user has certain behaviour or attributes. This behaviour relates to

a) what he is spending on,
b) with which merchant he interacts,
c) How much does he usually spend,
d) His activity trend over the years,
e) has he ever committed fraud?

And so on…All these behaviours would go as a user node's behavioural attributes. We make it simple. We collect aggregate over *Amount* a user generally spends. So, we find his average spending, minimum spending, maximum spending and variation in spending. To do that, we group by user and calculate all these aggregates (see below). Every node of the same type must have a label. Label can be, for example, 'User' or 'Member'. We keep it as 'member'. Also if a user has committed fraud even once, we say that all his transactions are 'suspect'.

User node must have two *must* fields: Id and Label. Besides these two, to distinguish Users from Merchants, we also have a '*Cat*' field. The '*Cat*' field has two values: *Member* for Users and *Institution* for Merchants.

Besides, these three field names, other fields may also be there and may carry any names.

If a user has committed fraud, ***EVEN ONCE***, we mark him with propensity to commit frauds. A column 'suspect' is added to records this propensity.

We group by user over Amount to get user node characteristics:

```
## 7.0 Group by user to get user node charteristics:
# StackOverflow:  https://stackoverflow.com/a/68726106/3282777
grpd_user = df_sample.groupby(['User'])
user_nodes = grpd_user.agg({'Amount' : [('u_min','min'),('u_max','max'),('u_mean','mean'),('u_std',np.std)] }).reset_index()
user_nodes = user_nodes.round(decimals = 2)
user_nodes.head()
```

	User	Amount			
		u_min	u_max	u_mean	u_std
0	u0	-255.0	1049.82	84.78	90.16
1	u1	-498.0	498.00	41.82	54.28
2	u10	-492.0	1067.21	44.98	89.07
3	u100	-289.0	766.36	100.45	122.21
4	u101	-302.0	302.00	40.78	47.84

Figure 5: Aggregate data set that gives user node properties.

Adding two fields: 'suspect' (a string field) and 'suspect_n' (boolean but numeric field):

```
# 7.9 We also create a column 'suspect_n' that records
#     1 for 'Yes' fraud and 0 for 'No' fraud
#     It is simply a numeric transformation of 'suspect' field:
#     We do this as data manipulation capabilities within gephi
#     are limited:

user_nodes['suspect_n'] = user_nodes['suspect']
user_nodes['suspect_n'] = user_nodes['suspect_n'].map({"Yes":1 , "No" : 0})
```

```
# 7.11
user_nodes.head()
```

	Id	Amount				suspect	Cat	Label	suspect_n
		u_min	u_max	u_mean	u_std				
0	u0	-255.0	1049.82	84.78	90.16	No	Member	u0	0
1	u1	-498.0	498.00	41.82	54.28	No	Member	u1	0
2	u10	-492.0	1067.21	44.98	89.07	No	Member	u10	0
3	u100	-289.0	766.36	100.45	122.21	No	Member	u100	0
4	u101	-302.0	302.00	40.78	47.84	No	Member	u101	0

Figure 6: User nodes data with per-node properties.

Save this file as '*users.csv*'.

Merchant Nodes:

For every merchant, we calculate, what is the minimum and maximum transaction amount. What is the mean transaction amount and its overall std deviation.

```
## 8.0 Group by merchant and get merchant charteristics:
grpd_merchant = df_sample.groupby(['Merchant Name'])
merchant_nodes = grpd_merchant.agg({'Amount' : [('m_min','min'),('m_max','max'),('m_mean','mean'),('m_std', 'std')]}).reset_index(
merchant_nodes = merchant_nodes.round(2)
merchant_nodes.head()
merchant_nodes.shape   # (10853, 2)
```

	Merchant Name	Amount			
		m_min	m_max	m_mean	m_std
0	m0	58.53	150.28	104.40	64.88
1	m1	18.14	18.14	18.14	NaN
2	m10	186.43	186.43	186.43	NaN
3	m100	18.96	18.96	18.96	NaN
4	m1000	4.82	5.26	5.00	0.19

Figure 7: Evaluating merchant properties

If a merchant has been victim of fraud ('Is Fraud' is Yes), we add a column to record this with 'victim' as 'Yes' else 'No'. And create a column 'victim_n' as 0 or 1. Also add a category to it as '*Institution*'.

```
# 8.1 With which one of the merchants fraud comitted:
df_sample.loc[df_sample["Is Fraud?"] == "Yes", 'Merchant Name' ].unique()
merchant_suspect = df_sample.loc[df_sample["Is Fraud?"] == "Yes", 'Merchant Name' ].unique()
len(merchant_suspect)
```

```
# 8.2 We record this also in 'victim':
merchant_nodes['victim'] = "No"

# 8.3 Merchant 'Cat' we designate as 'Institution'
merchant_nodes['Cat'] = 'Institution'
```

```
# 8.3 Record which all merchants have propensity to become victims:

for i in merchant_suspect:
  merchant_nodes.loc[merchant_nodes['Merchant Name'] == i, 'victim' ] = "Yes"
```

```
# 8.4
merchant_nodes['victim'].value_counts()  # 10777, 76
```

Figure 8: A column victim records if a merchant was victim of fraud.

Our final merchant's data:

```
# 8.5.1
merchant_nodes.head()
merchant_nodes.shape   # (10853, 6)
```

	Id	Amount				victim	Cat	Label	victim_n
		m_min	m_max	m_mean	m_std				
0	m0	58.53	150.28	104.40	64.88	No	Institution	m0	0
1	m1	18.14	18.14	18.14	NaN	No	Institution	m1	0
2	m10	186.43	186.43	186.43	NaN	No	Institution	m10	0
3	m100	18.96	18.96	18.96	NaN	No	Institution	m100	0
4	m1000	4.82	5.26	5.00	0.19	No	Institution	m1000	0

(10853, 9)

Save this file as '*merchants.csv*'.

Edges

Edges must have at the least three fields: *Source*, *Target* and *Type*. *Type* field records if an edge is *directed* or *undirected*. Besides these three, it may have other field-properties also. For example, we can have edge-weight as being number of interactions between user and the merchant. Here is edge-weight calculations:

```
# 9.0 Group by User and Merchant Name:
#     We also wish to record the number of interactions between
#     customer and merchants. These will be edge weights:

edges = df_sample.groupby([df_sample['User'], df_sample['Merchant Name']]).size()
```

```
# 9.1 The size column has a name '0'. We need to rename it:

edges = df_sample[['User', 'Merchant Name']].groupby(['User', 'Merchant Name']).size().reset_index()
edges = edges.rename(columns = { 0 : "weight"})
edges.head()
```

	User	Merchant Name	weight
0	u0	m10039	3
1	u0	m10101	5
2	u0	m10183	1
3	u0	m10800	2
4	u0	m1118	5

Figure 9: Edge weights

We rename the first two columns as *Source* and *Target*.

```
# 9.2 We rename the first two columns appropriately:

edges = edges.rename(columns = {'User': 'Source', 'Merchant Name': 'Target'})
```

```
# 9.3 Our edges are Undirected:

edges['Type']= 'Undirected'
edges.head()
```

	Source	Target	weight	Type
0	u0	m10039	3	Undirected
1	u0	m10101	5	Undirected
2	u0	m10183	1	Undirected
3	u0	m10800	2	Undirected
4	u0	m1118	5	Undirected

Figure 10: Edge weights and edge types

To get other fields, we group users and merchants and aggregate certain properties. Save this file as '*edges.csv*'.

Back to gephi

Import User nodes

Import user nodes from file: *creditCard_users.csv*. To import, click **File→Import spreadsheet**. While importing, check that all data-types of features being imported are correct and also uncheck '*Create missing nodes*' option. At times this option may be greyed out. IF so, do not worry. But after edges have merged, if missing nodes are created by gephi on its own, check *Data Laboratory*. *Created missing nodes would have blank labels*.

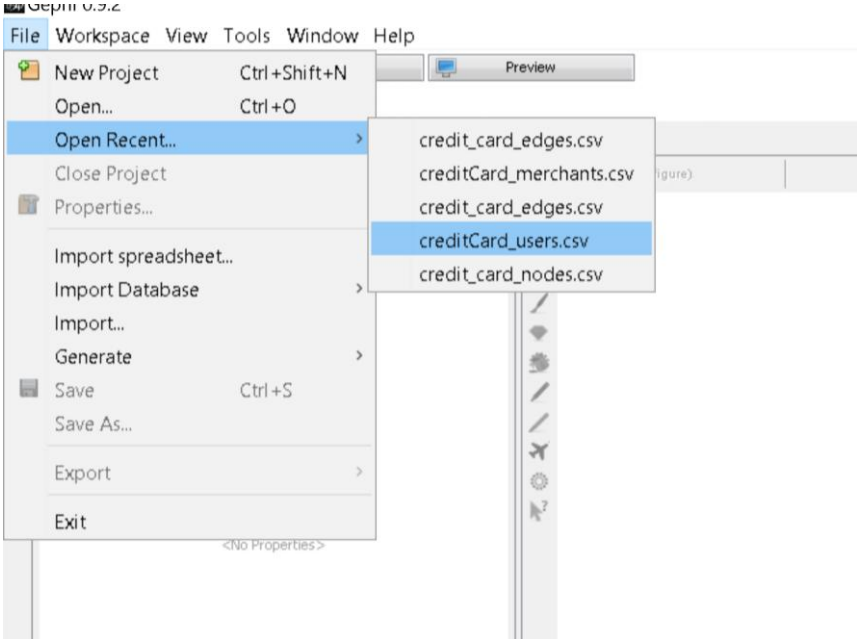


Figure 11: To import a csv file, click File->Import spreadsheet.

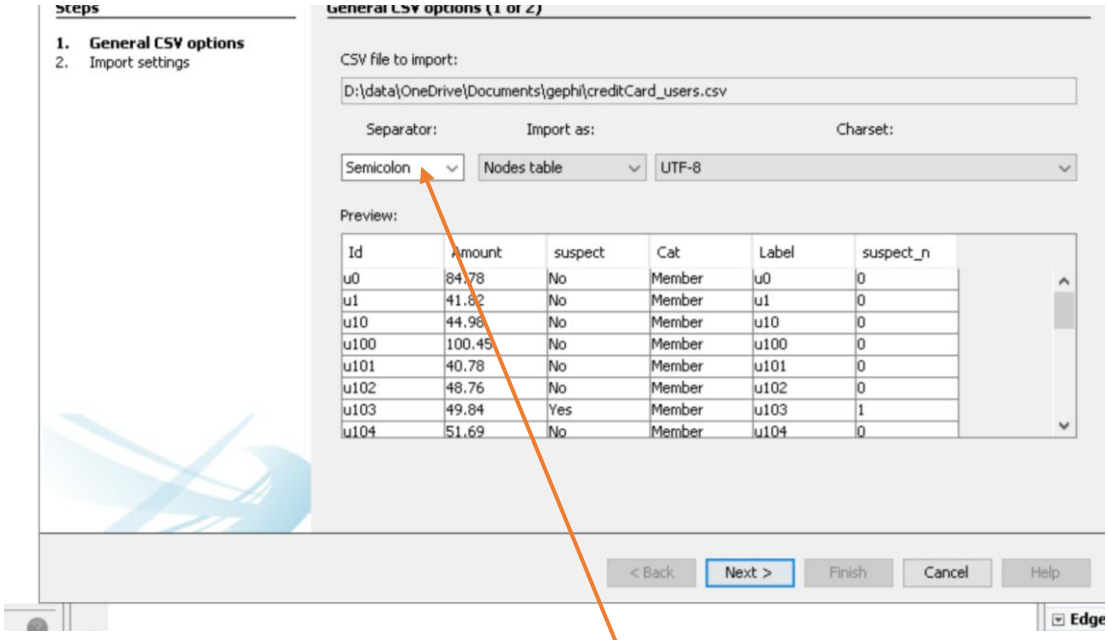


Figure 12: Check that separator is correct

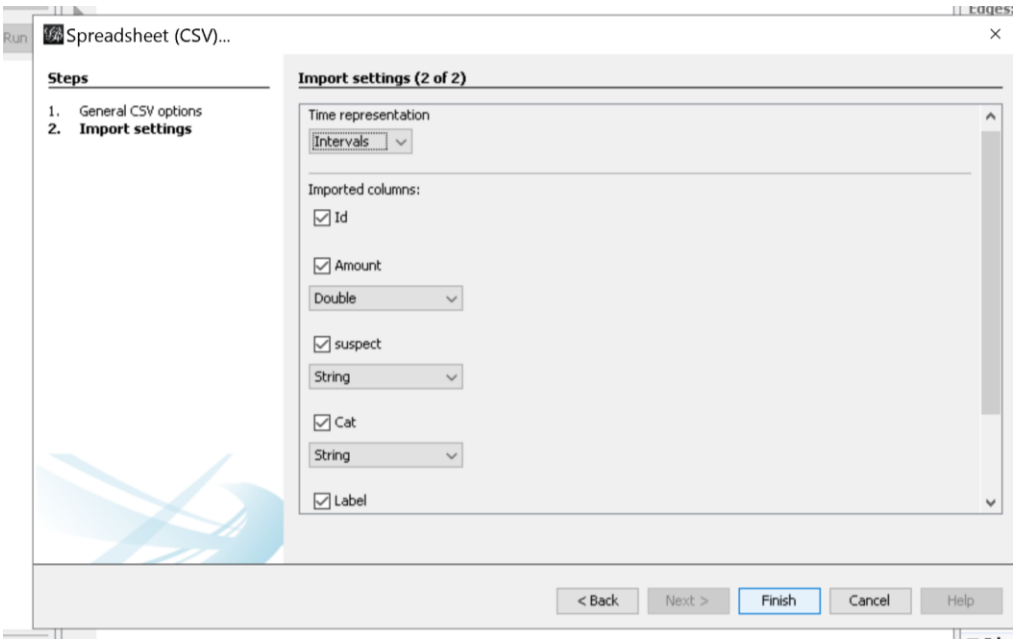


Figure 13: Check that all datatypes are taken as correct. Else, change them.

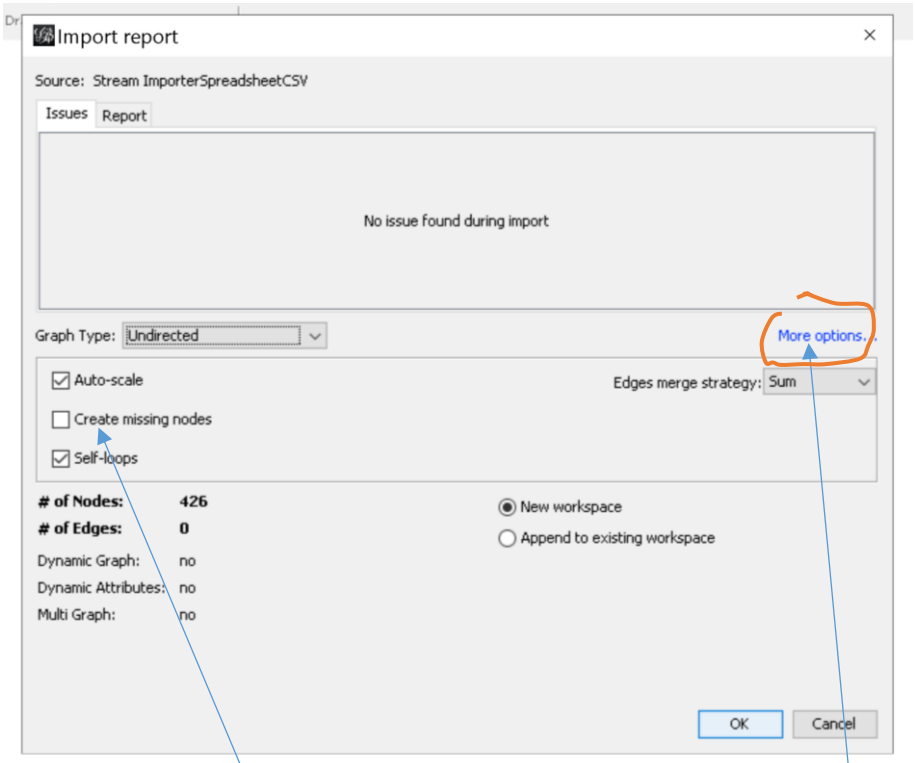


Figure 14: Uncheck 'create missing nodes option'. It may also be greyed out. You may have to click 'More options' link to view them.

Import Merchant nodes

Import as spreadsheet (*File→Import spreadsheet*), the file: *creditCard_merchants.csv*. You have to '*Append to existing workspace*' and also Do not create any missing edges.

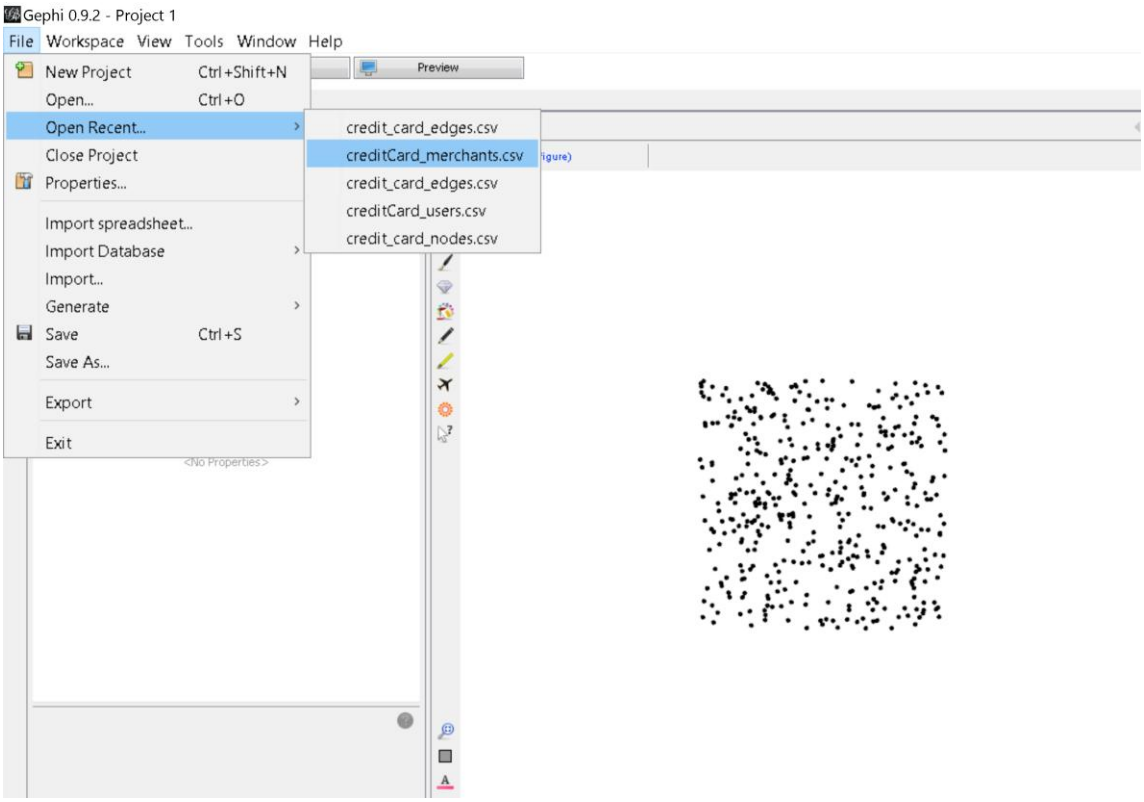


Figure 15: Import file creditCard_merchants.csv as spreadsheet

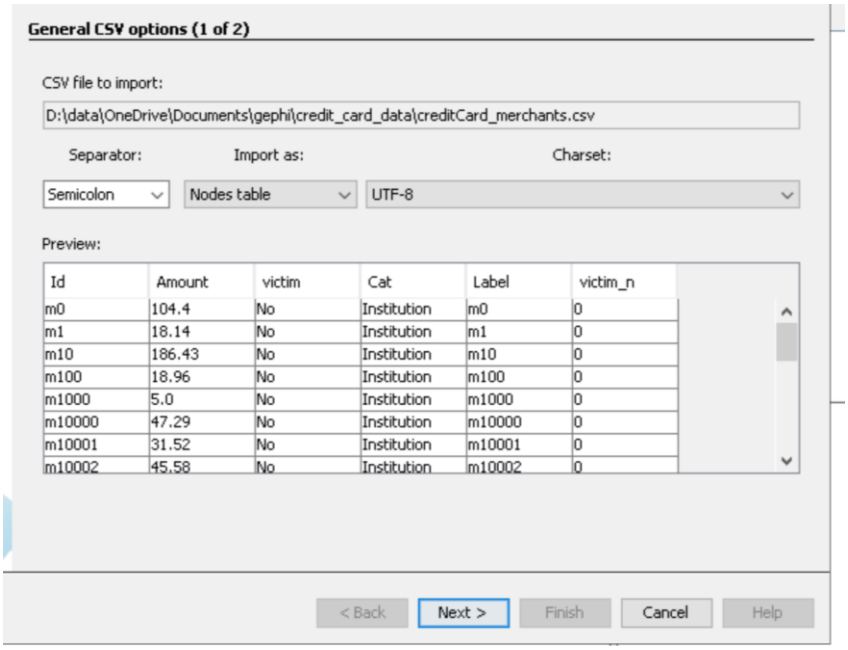


Figure 16: Check separator

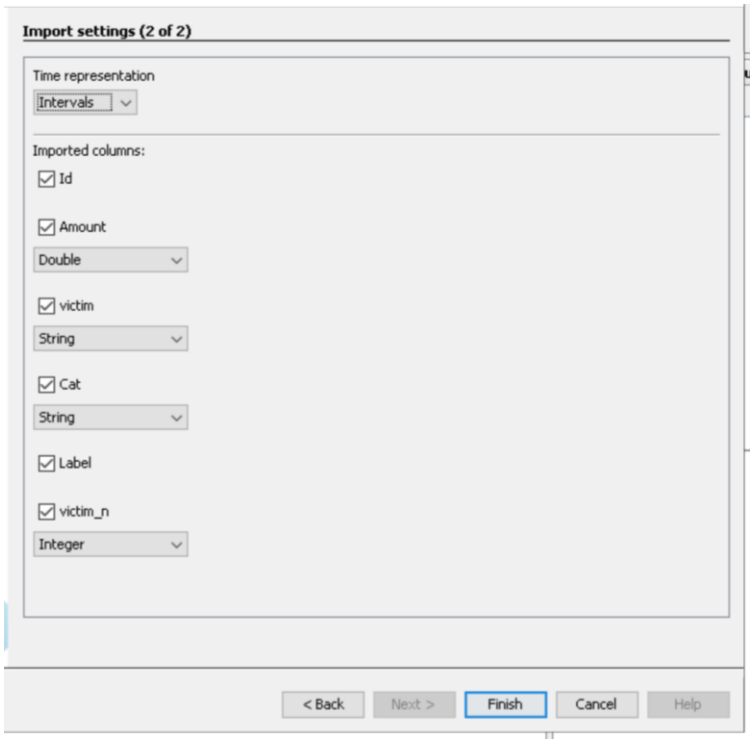


Figure 17: Check datatypes

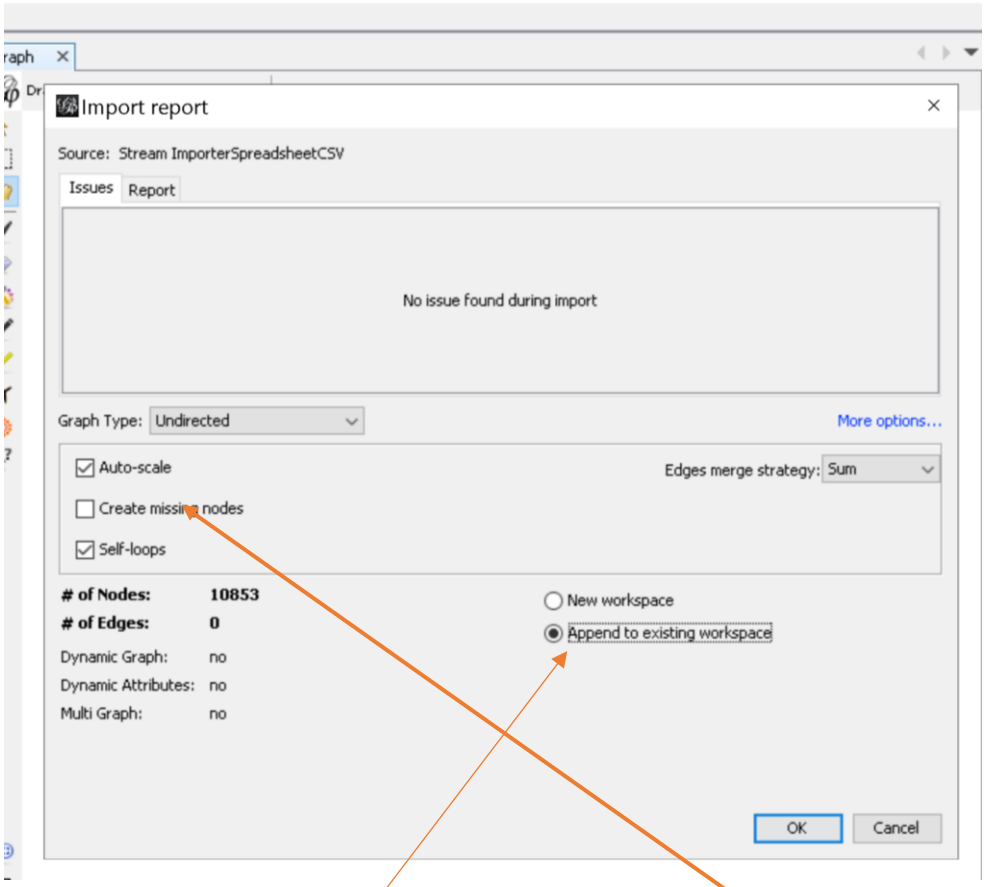


Figure 18: Select 'Append to existing workspace' and uncheck create missing nodes

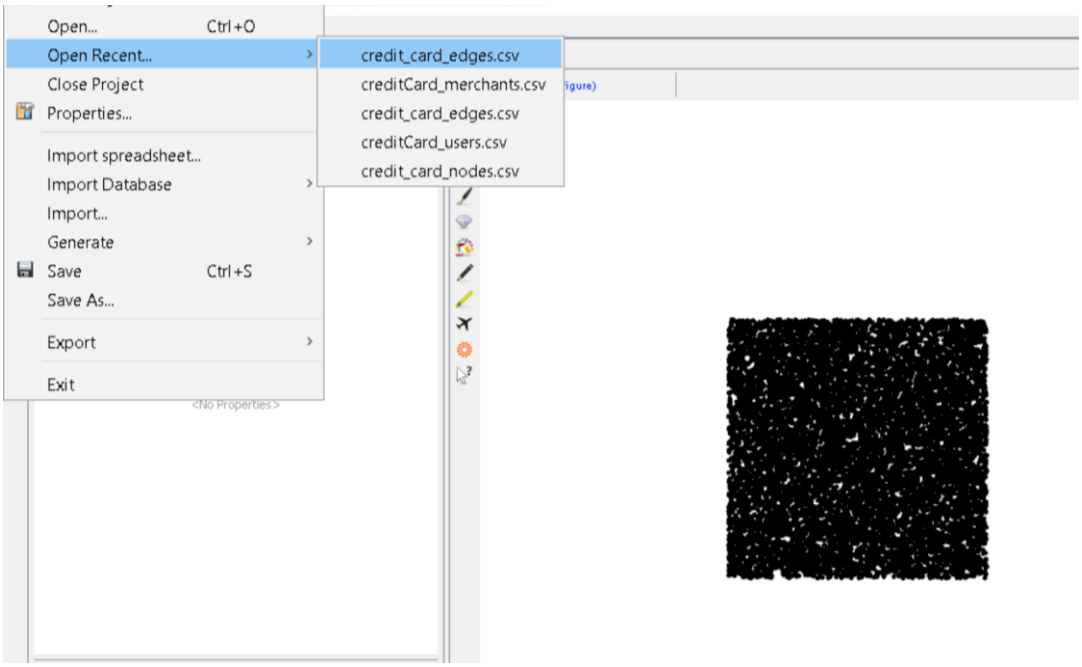


Figure 19: Imported nodes. Next, import edges

Import edges

Import file *creditCard_edges.csv* as spreadsheet. Import this file in exactly the same manner as you imported nodes for merchants.

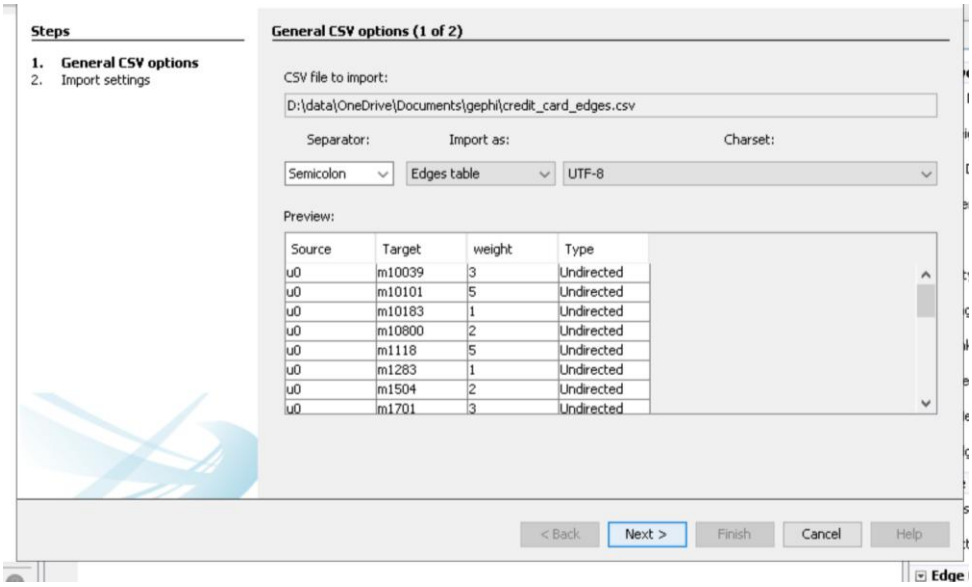


Figure 20: Edges import--I

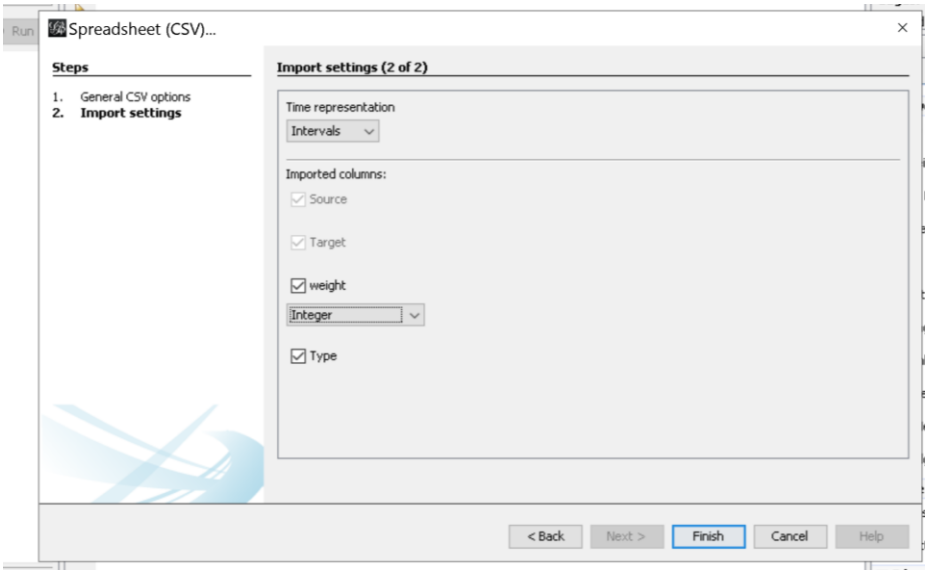


Figure 21: Edges import—II. It is up-to you to import 'Weight' or not. As weight here denotes interactions, better import them. Weight import complicates calculations. We will keep it simple.

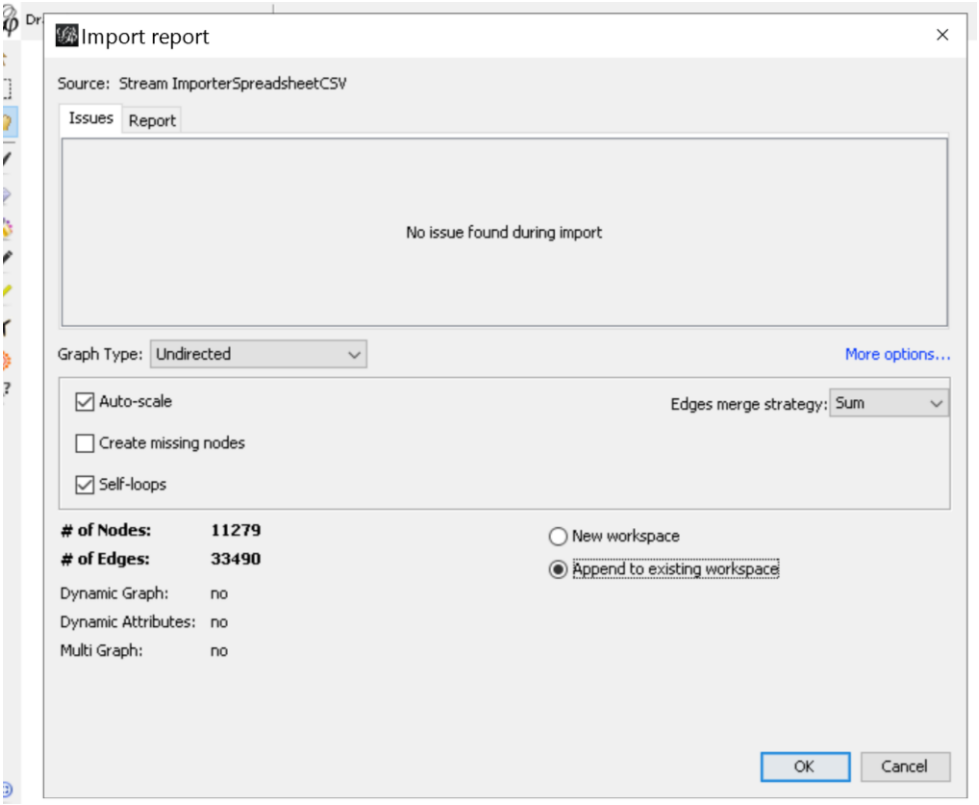


Figure 22: Edges import—III

Graph processing:

Select from *Layout* dropdown, option *Fruchterman Reingold* option with parameters as mentioned in the figure below. We set '*Area'* to 30000 and '*Speed'* to 30 as number of nodes are very large. Large speed sacrifices precision. Click '*Run'*. Wait for at the least for three minutes.

Fruchterman Reingold Area 30000, Speed: 30
3 minutes

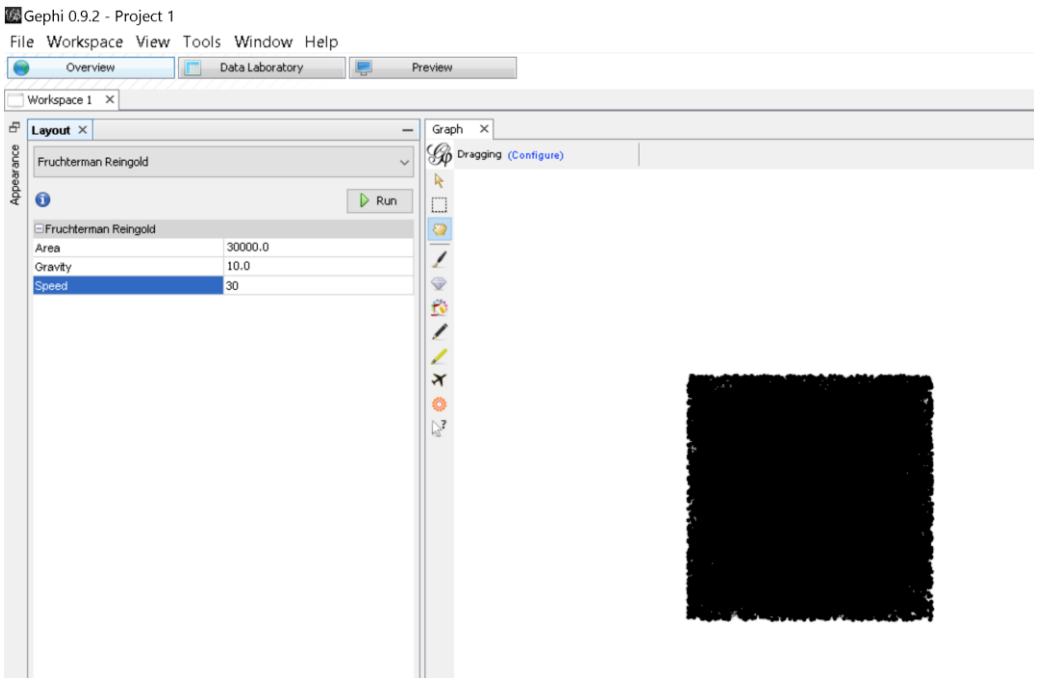
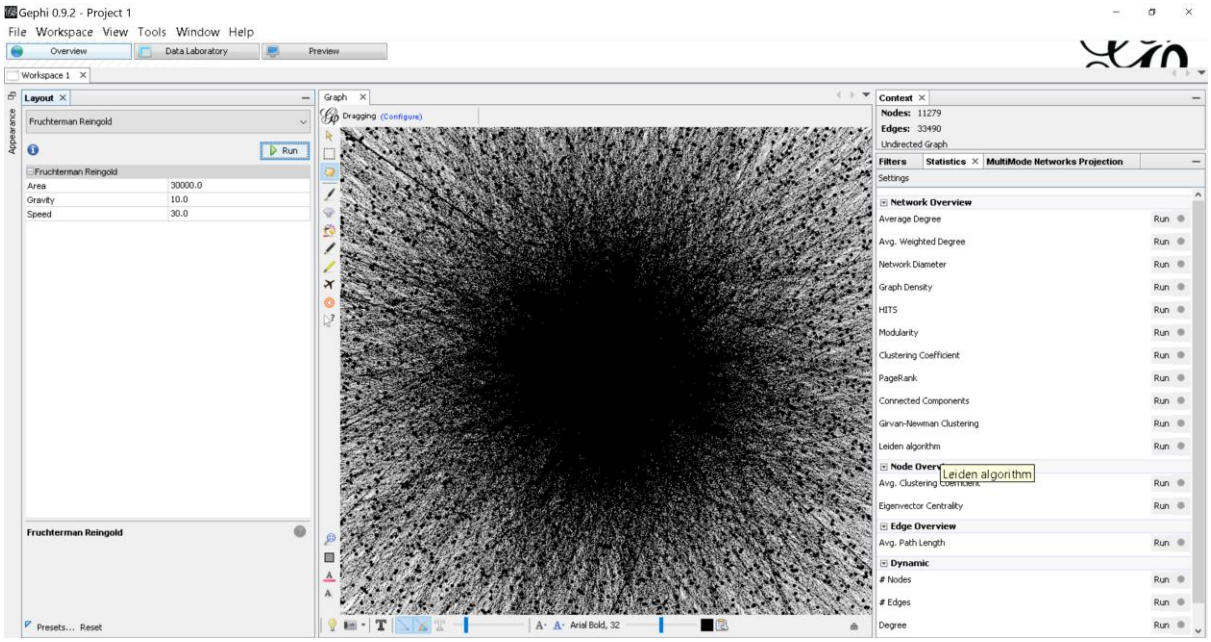


Figure 23: For Layout option: Fuchterman Reingold, set parameters as above. Click 'Run'. Wait for at the least 3-minutes.

Here is the graph after 3 minutes



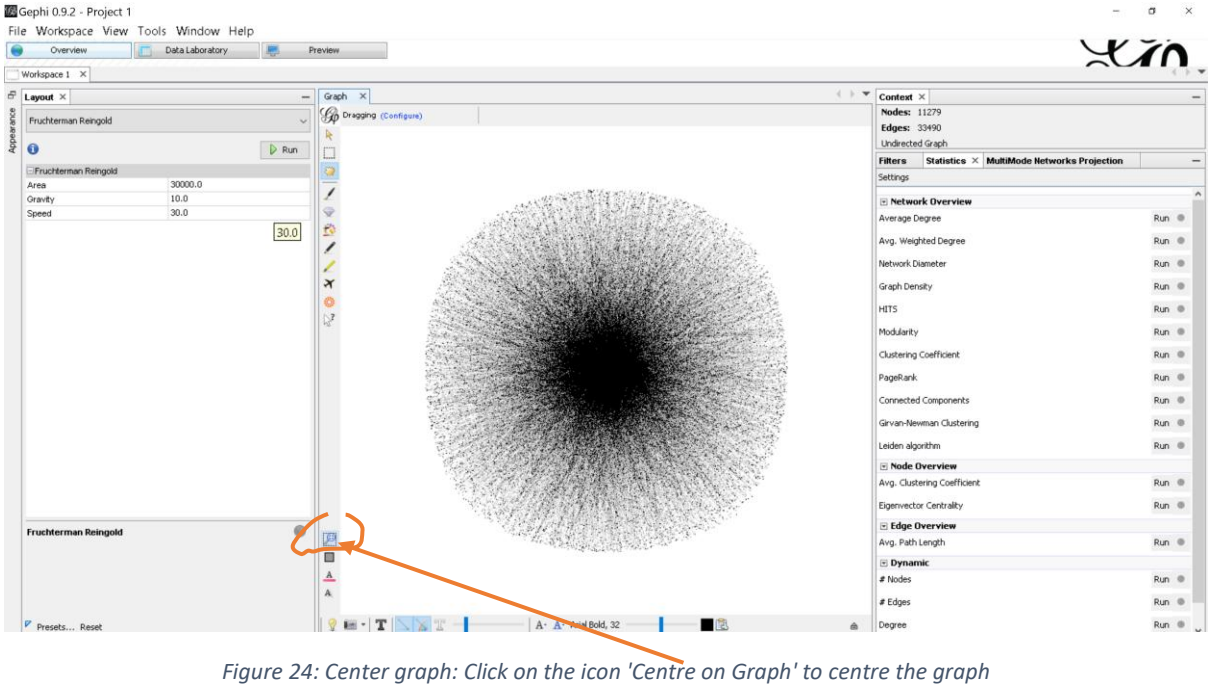


Figure 24: Center graph: Click on the icon 'Centre on Graph' to centre the graph

II

Force Atlas 2:
Scaling: 100
Prevent Overlap: Yes
5 minutes

Next, select *Layout→Force Atlas2*. Set *Scaling* to 100. Scaling decides, how much repulsion does one want. More scaling makes a more sparse graph. Also check *Prevent Overlap*.

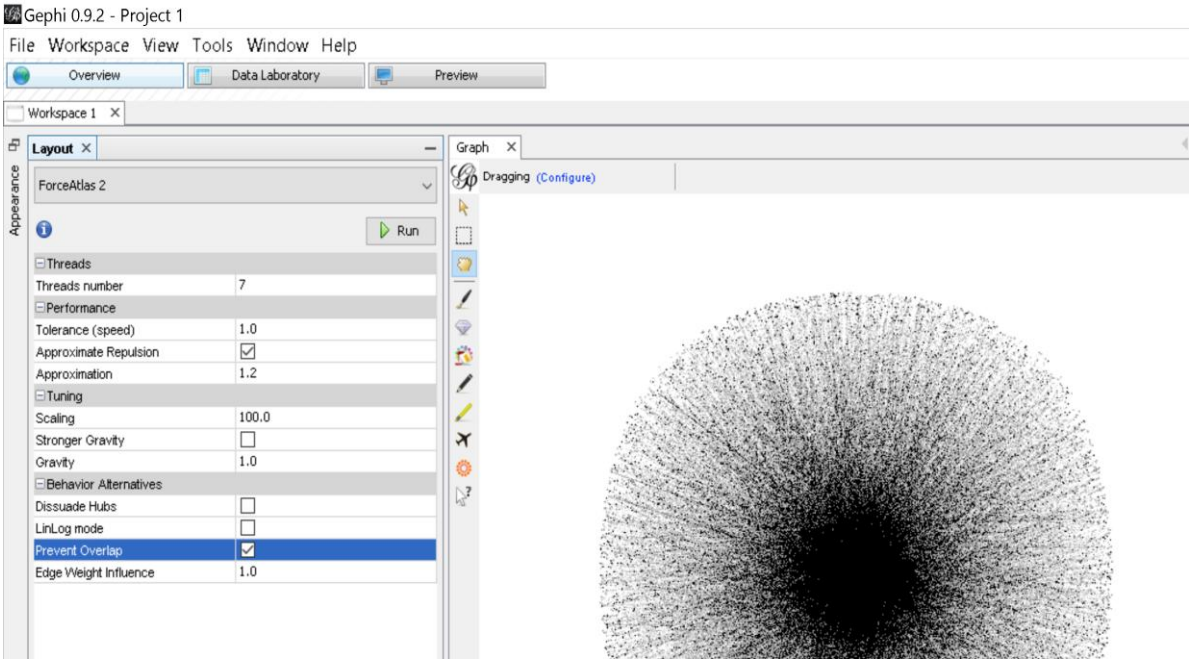


Figure 25: Change Scaling value to 100 and check mark '<u>Prevent Overlap</u>'. Click run and wait for 5-minutes

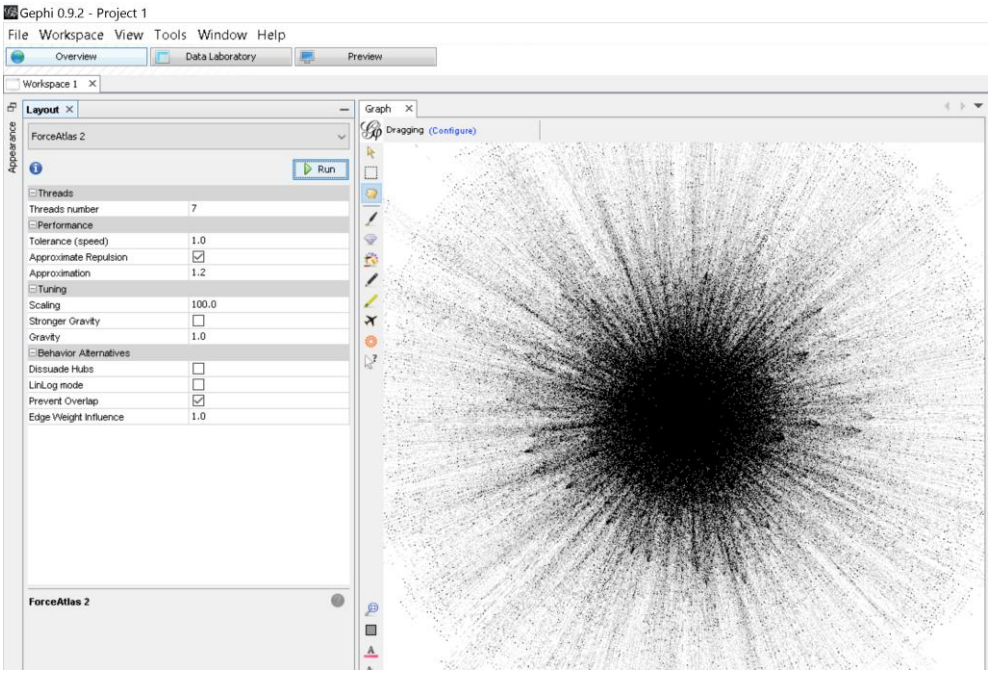


Figure 26: after 5-minutes

III Differentiating Users and Merchants nodes:

Click on *Nodes→Colour icon-> Partition.* Select '*Cat'* from dropdown. Click *Apply* button.

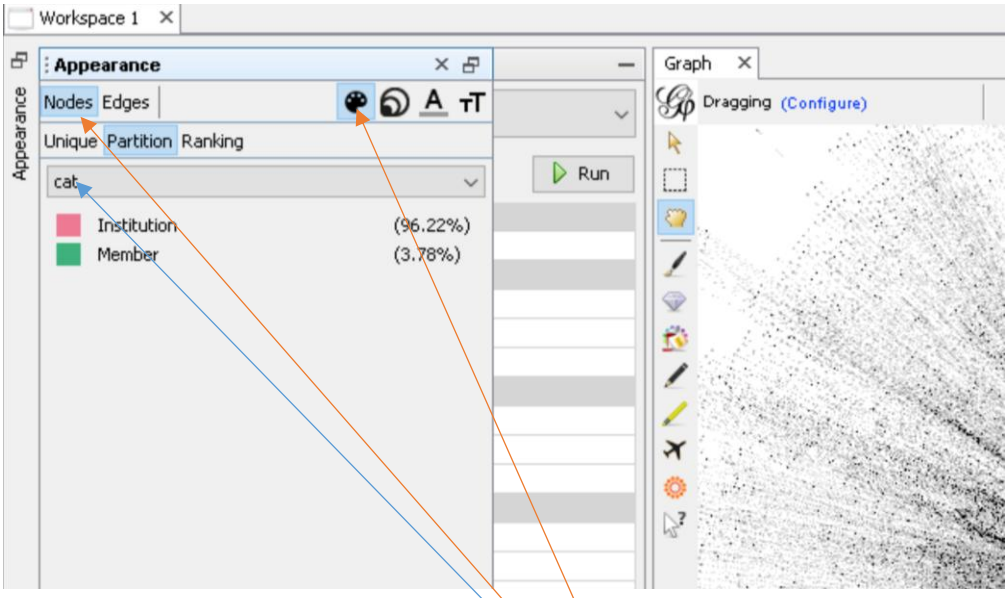


Figure 27: Petitioning nodes into Users vs Merchants: Click Nodes, color icon and then Partition, In the Partition dropdown, select Cat.

IV: Differentiating 'Is Fraud' vs no-fraud nodes

Change node size as per 'is Fraud' or not.

suspect_n
Ranking
Min size: 30
Max size: 150

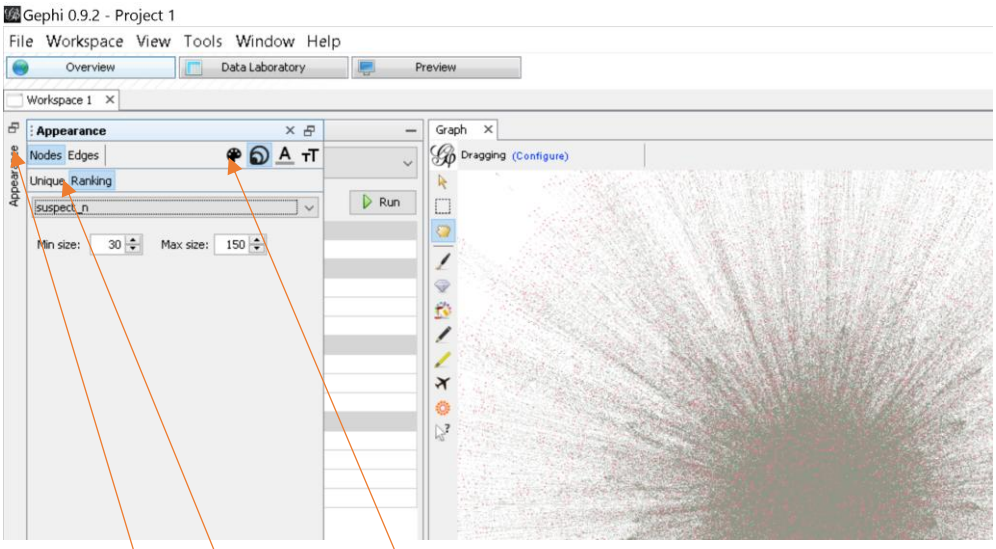


Figure 28: Further, select Nodes →Ranking and select Size icon. In the drop-down select 'suspect_n'. Set Min node size to 30 and Max node size to 150. Click Apply.

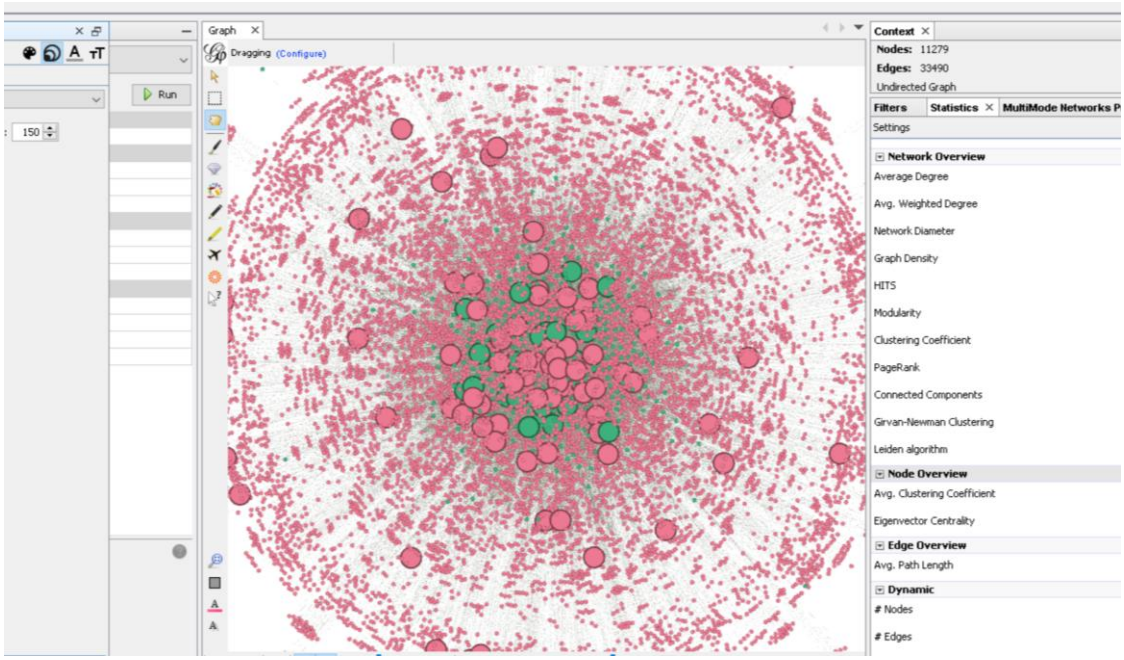
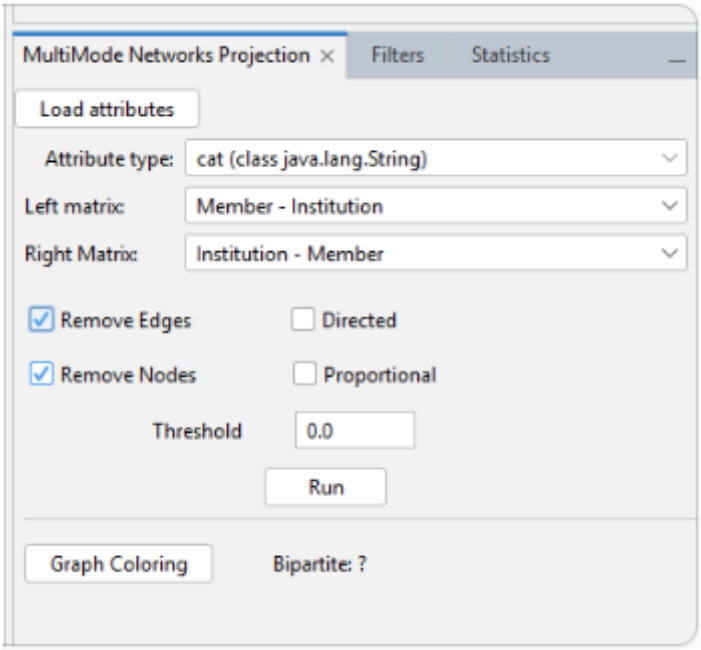


Figure 29: Resulting graph after above changes.

V: Projecting two mode graphs to one mode

We project the two-mode graph to one mode. A two-mode graph has two types of nodes. By projecting to one we eliminate one of the 'in-between' nodes and make (so-to-say) direct connections.

Open tab Multimode Networks for Projection. Click first on button *Load Attributes*. Then make selections exactly as made here. Attribute type is '*Cat*'. We have to select left and right matrix, as: *Member-Institution* (left) and *Institution-Member* (right) and check mark *Remove Edges/Remove Nodes*. These remove extra nodes/edges. Click *Run* button. Wait for some time. It takes time.



This is the Gephi MultiMode Networks Projection plugin. Here's what "Remove Edges" does, and your hunch is partly right but needs a correction.

What "Remove Edges" does

When you run a projection, the plugin multiplies the two matrices (Left × Right). Here, Member–Institution times Institution–Member gives a Member–Member network, where two members are linked if they share one or more institutions.

"Remove Edges" deletes the *original* bipartite edges (the Member–Institution links) after the projection is computed. It doesn't merge duplicates. So you end up with only the new projected edges rather than a mix of old and new ones.

"Remove Nodes" works the same way: it deletes the nodes of the type that got projected away (the Institution nodes), so you're left with a clean one-mode Member–Member network.

Edge weights and duplicates

The weight-increasing behavior comes from the matrix multiplication itself, not from this checkbox. If two members share three institutions, the product gives a value of 3 for that pair, and that becomes the edge weight. Each pair gets one edge, with the weight reflecting the number of shared neighbours. So duplicates are consolidated into a single weighted edge as a natural result of the projection, regardless of whether Remove Edges is checked.

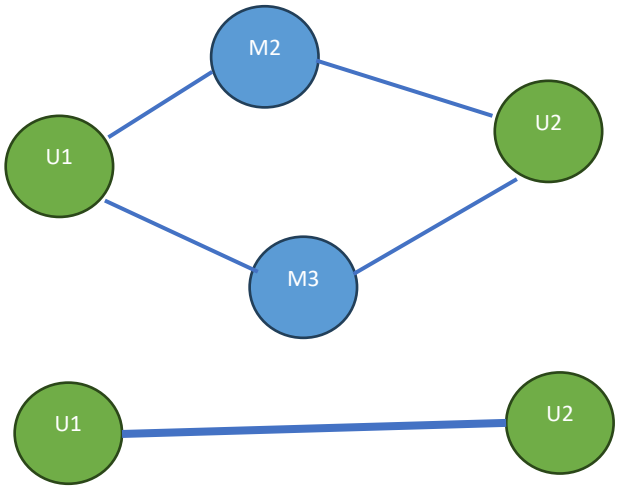
Related options in your screenshot

- **Proportional** (unchecked): would normalize the weights instead of using raw counts.
- **Threshold** (0.0): edges with weight at or below this value are dropped, so raising it filters out weak connections.
- **Directed** (unchecked): produces undirected projected edges.

Practical tip: if you want to keep your original bipartite network for comparison, uncheck both Remove Edges and Remove Nodes, or duplicate the workspace before running.

Bimodal Projection

We will find out the connections between users that have high 'edge weights'. Conceptually bimodal projection does the following. In the first figure we have two merchants and two users. Edge weights are 1. Bimodal projection connects users directly, eliminating merchants. But now the edge has twice the earlier weight.



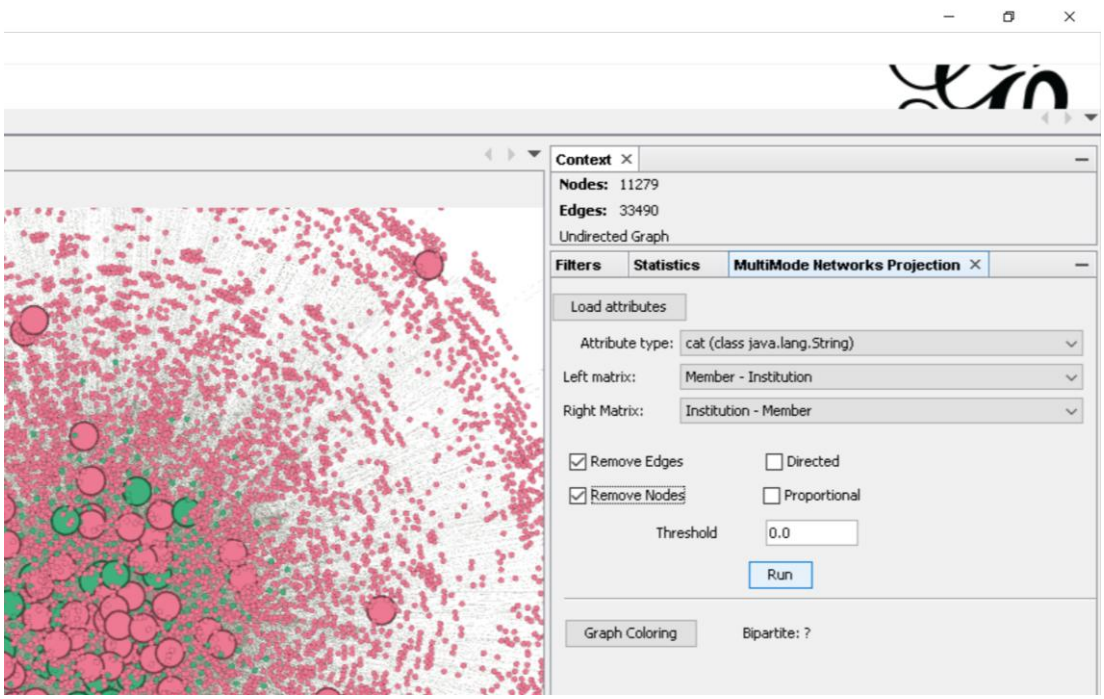


Figure 30: Click first Load Attributes. Then make selections EXACTLY as made here. 'Member-Institution' AND 'Institution-Member' on Run and wait for some time. The result will be as below.

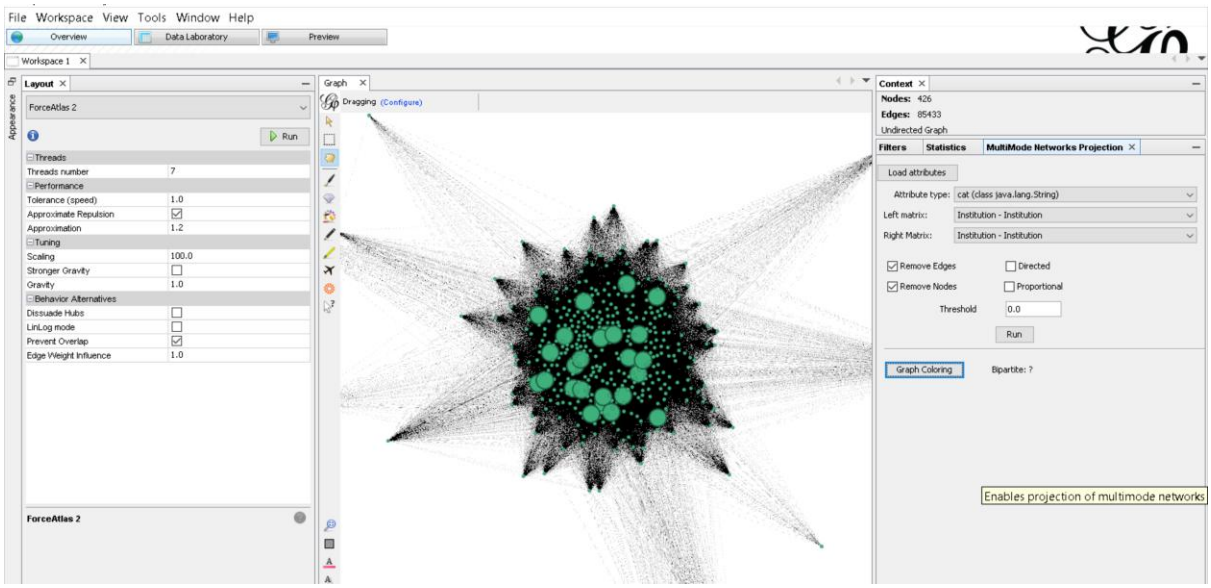


Figure 31: Brick color is gone. Only Green is left.

Analysing network with filters and communities:

Plan

We will filter our data first by *Amount*. Generally, frauds will not occur when the amount is small. We will also filter data when '*suspect_n*' is 1 i.e. consider only nodes that have committed fraud. After all our intention is to discover a community of fraudsters. We also mask *Edges* by *weight*. A community of fraudsters will have more interaction between them. We get certain nodes connected with high weight edges. We also filter our data by Modularity class to check if the highly weighted edges belong to a certain community.

Interpretation of this result can be seen at Interpretation of Filtered Graph. Our overall filters would be as below and in this order:

I. Range filter: Amount >=20
II. Partition filter: suspect_n = 1
III. Edges: Edge Weight > 37.2
IV. Partition filter: Modularity Class

We will create nested filters.

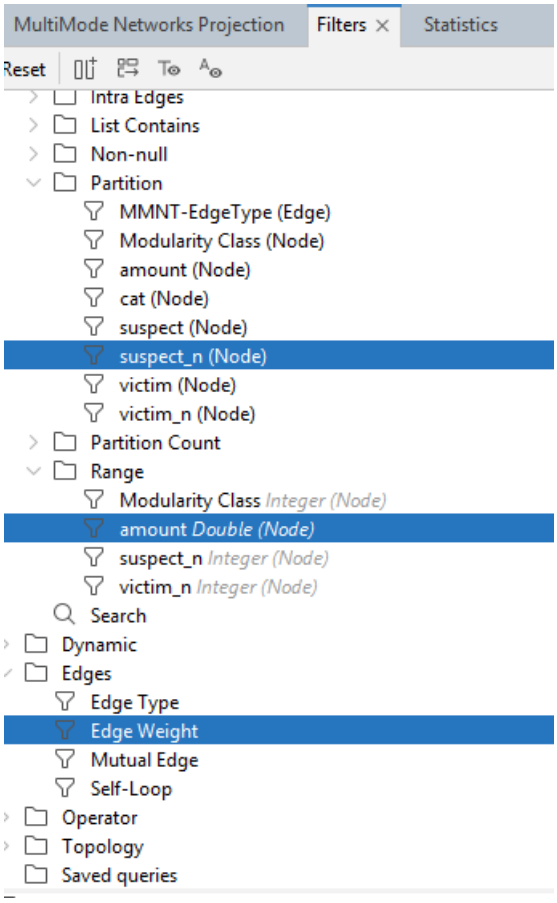


Figure 32: Filter scheme: From Range (amount), from Partition (suspect_n) and Edges (Edge Weight)

Nested Query window:

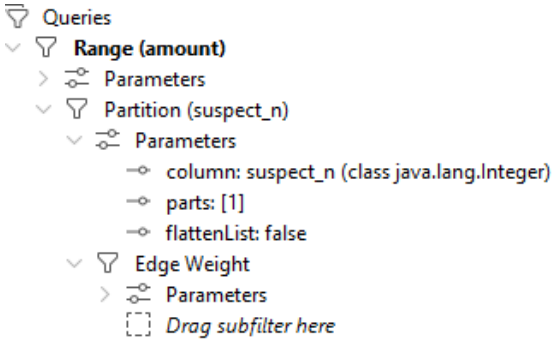


Figure 33: Nested query window

Modularity

Before we begin filtering data, let us try to divide our graph into four-structures using Modularity. In the *Statistics* tab, click on *Modularity*. Repeated clicking on *Modularity* may give you different number of communities. At 0.173. we get four communities. Keep communities to minimum.

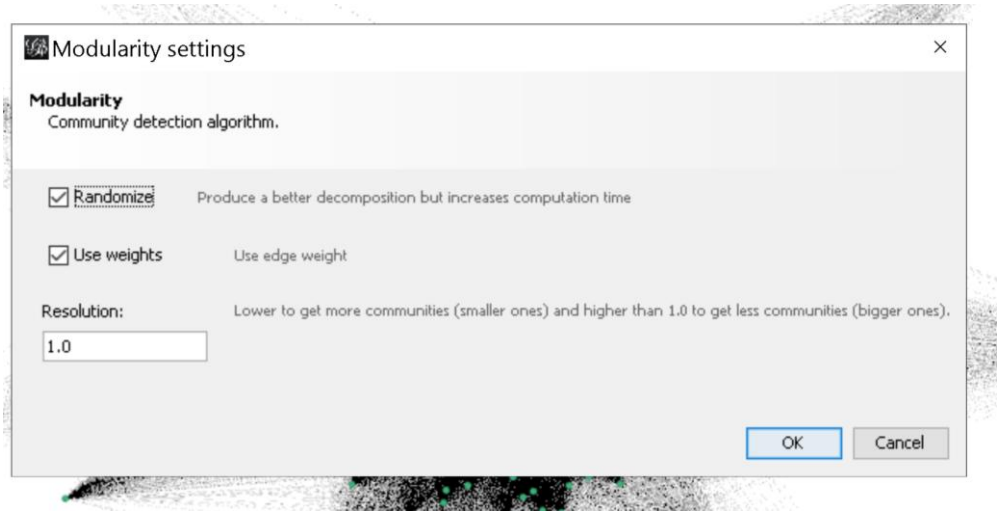


Figure 34: Repeated clicking on modularity may give you different number of clusters.

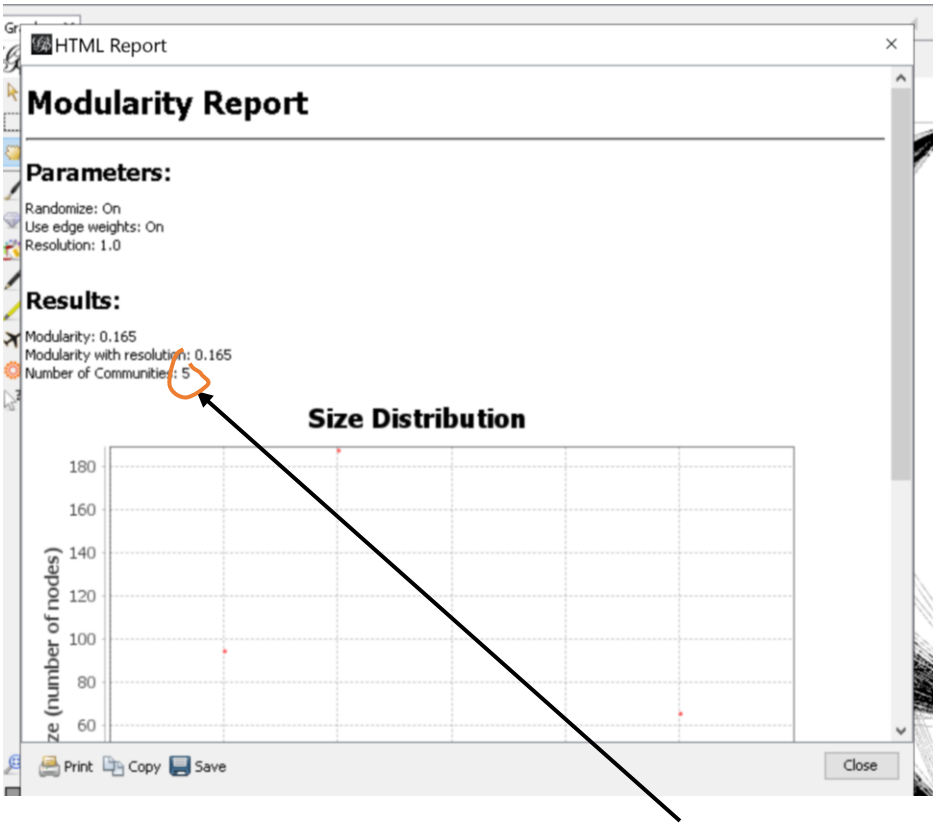


Figure 35: An illustrative modularity report (Results) with 5 communities.

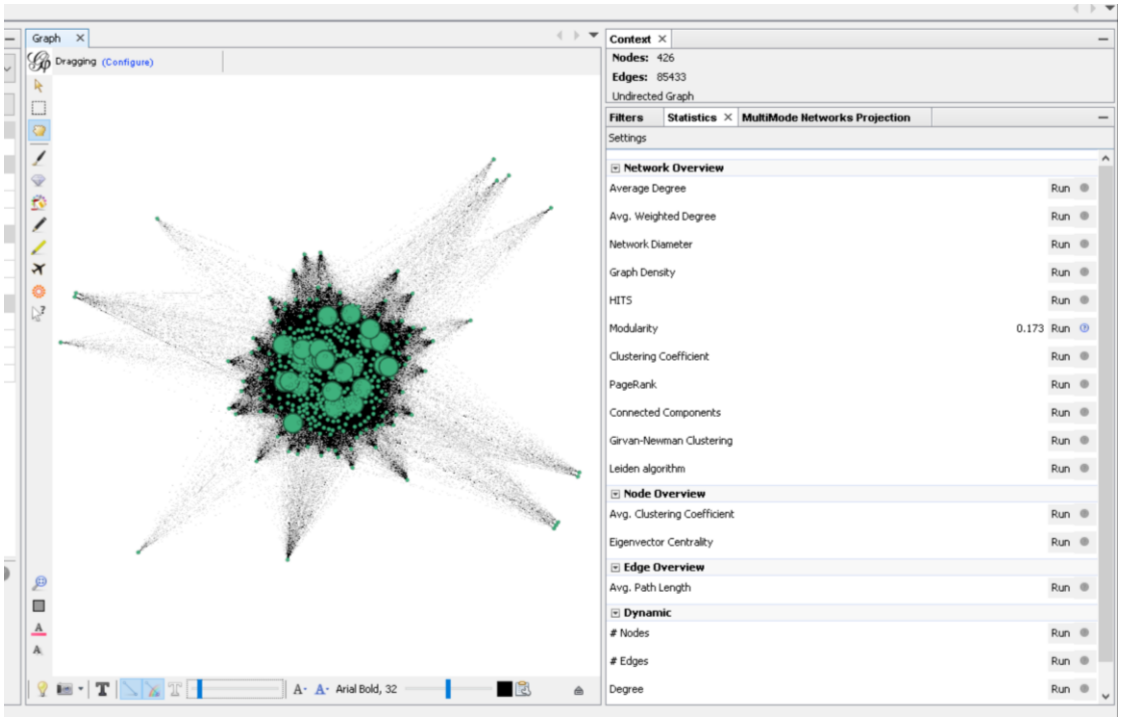


Figure 36: Modularity score of 0.173

Colour nodes as per community class. Two communities (colour magenta and green) dominate.

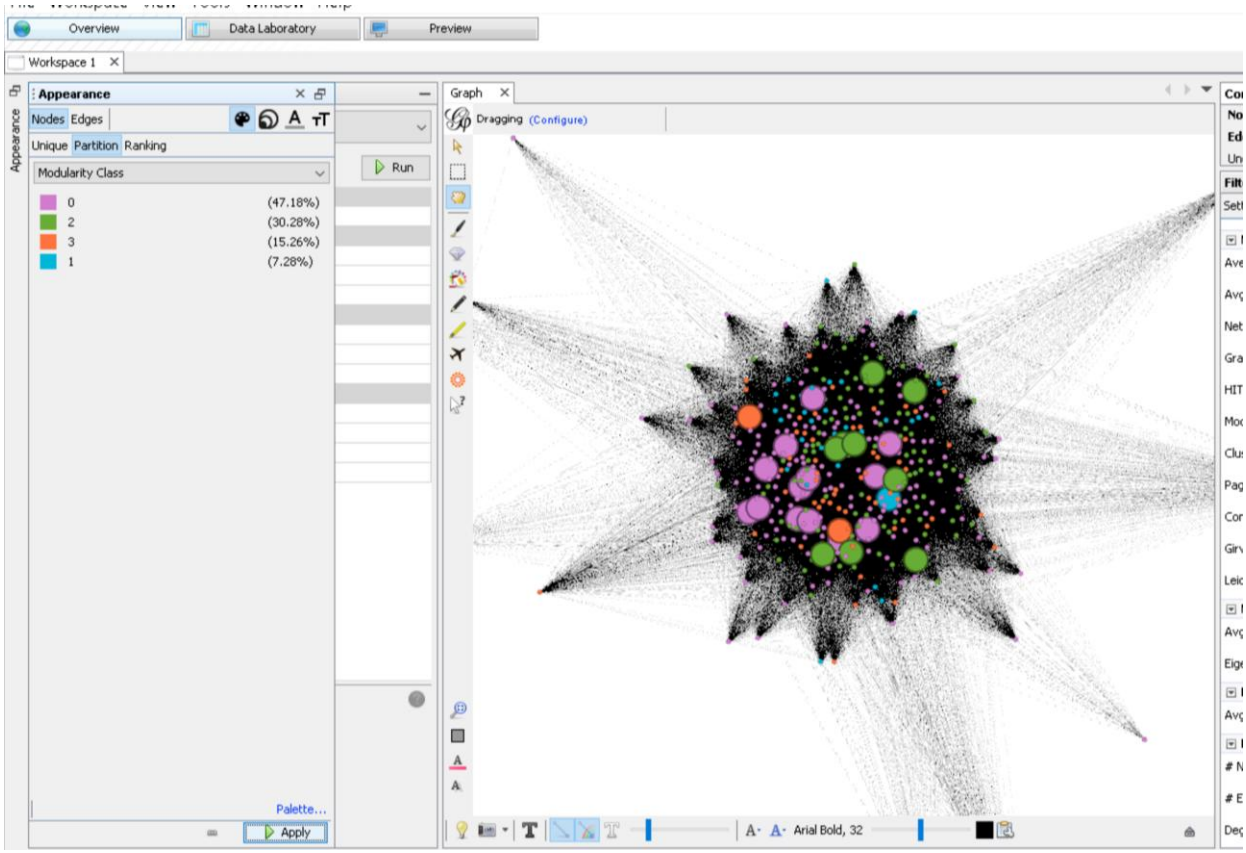


Figure 37: Click on Nodes, Partition, colour icon and in the drop-down select Modularity class. Click Apply button. Two communities dominate

At this point, click *File->Save* and save your work as *credit_card.gephi*

Remove blackness: Make edges lighter in color to have a better graph.

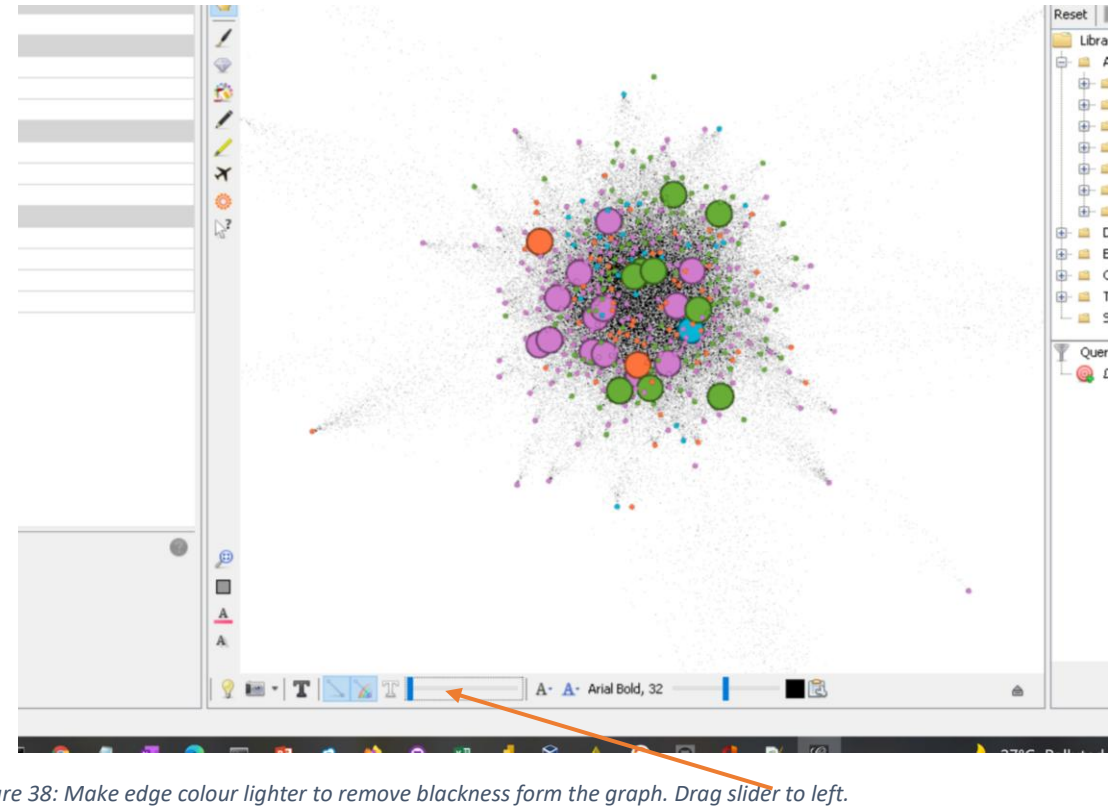


Figure 38: Make edge colour lighter to remove blackness form the graph. Drag slider to left.

Attributes filter?

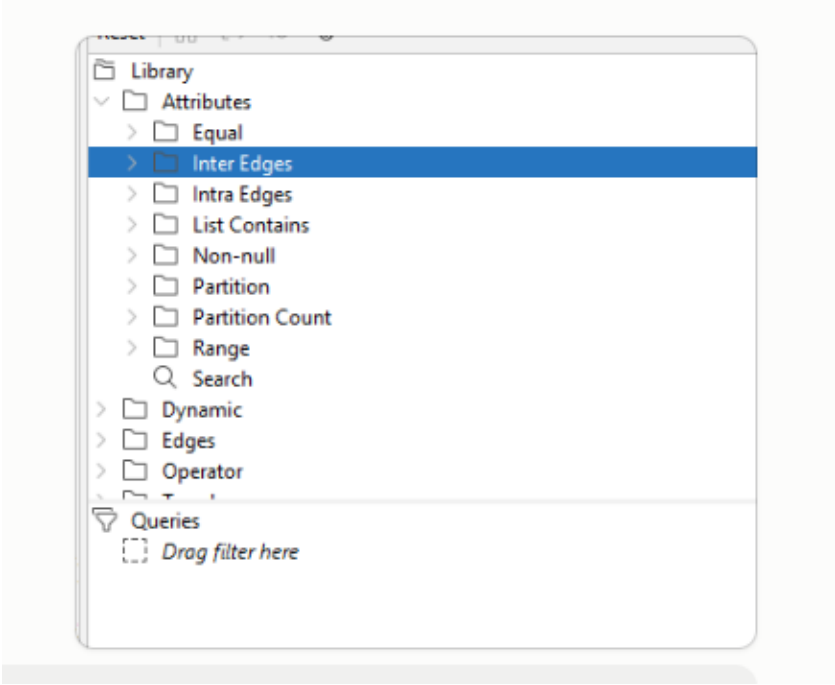


Figure 39: Under Attributes we have different types of filters. The implications of each are given below.

Here's what each filter under **Attributes** does in Gephi's Filters panel. All of them filter nodes or edges based on their data columns (attributes), and you use them by dragging into the "Queries" area and setting parameters.

Equal: Keeps only nodes/edges whose attribute exactly matches a value you specify. It has sub-types depending on column type (string, integer, float, boolean). For example, keep only nodes where Country = India, or Degree = 5.

Inter Edges: Keeps only edges that connect nodes from *different* groups, where groups are defined by the attribute you choose.

For example, if nodes have a "Community" attribute, Inter Edges shows only edges that bridge between communities. This is useful for finding connections across clusters.

Intra Edges: The opposite of Inter Edges. Keeps only edges that connect nodes with the *same* attribute value. Using the Community example, it shows only edges inside communities, which helps you see the internal structure of each cluster.

List Contains: Works on attributes that hold lists of values (like a "Tags" column containing several entries). It keeps nodes/edges where the list contains a particular value or pattern.

Non-null: Keeps only nodes/edges that have a value in the chosen attribute column, removing those with empty or missing data. Handy for cleaning incomplete datasets.

Partition: Filters by categories of a column. You select one or more values (for example, specific communities or modularity classes) and the graph keeps only nodes belonging to the selected ones. It's often used after running community detection.

Partition Count: Filters by how many distinct partitions a node's neighbourhood touches. For example, keep nodes that are connected to at least 3 different communities, which highlights "bridge" or broker nodes.

Range: Filters numeric attributes using a slider with min and max values. For example, keep nodes with Degree between 10 and 50, or PageRank above a threshold. This is probably the most commonly used filter for numeric data.

Search: (Shown with the magnifier icon.) Lets you search nodes/edges by text or regular expression within a chosen column, such as finding all nodes whose label contains "John".

Quick usage tip: Expand any folder (click the arrow), then drag the specific sub-filter into the **Queries** box at the bottom. Set its parameters, then click **Filter** to apply it. You can also combine multiple filters using the Operator folder (AND, OR, NOT).

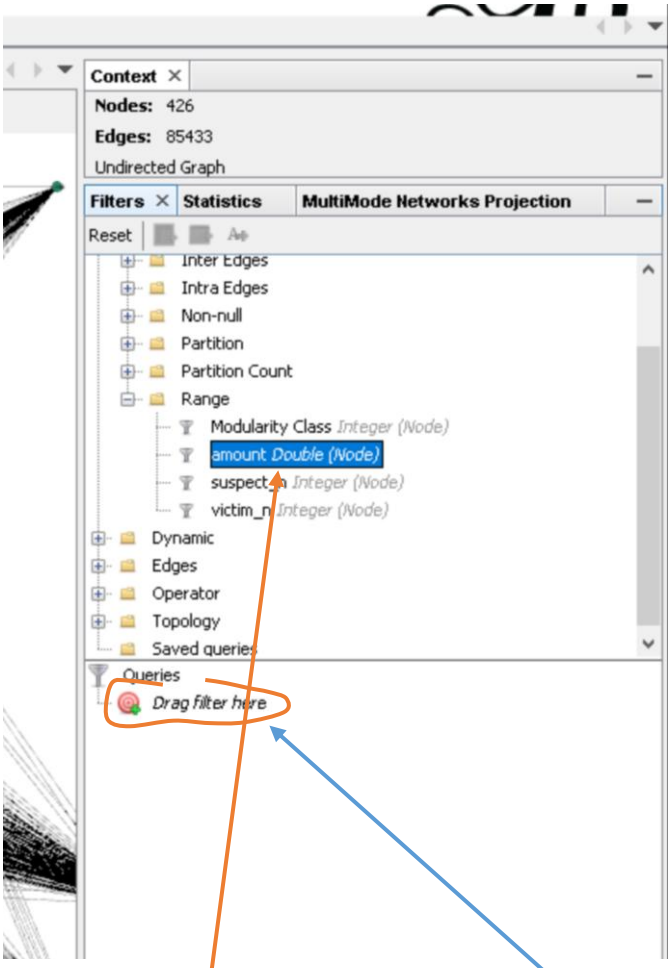


Figure 40: In Filter tab, drag Range-->Amount (or u_mean) over the line: Drag filter here in the lower panel (Queries panel).

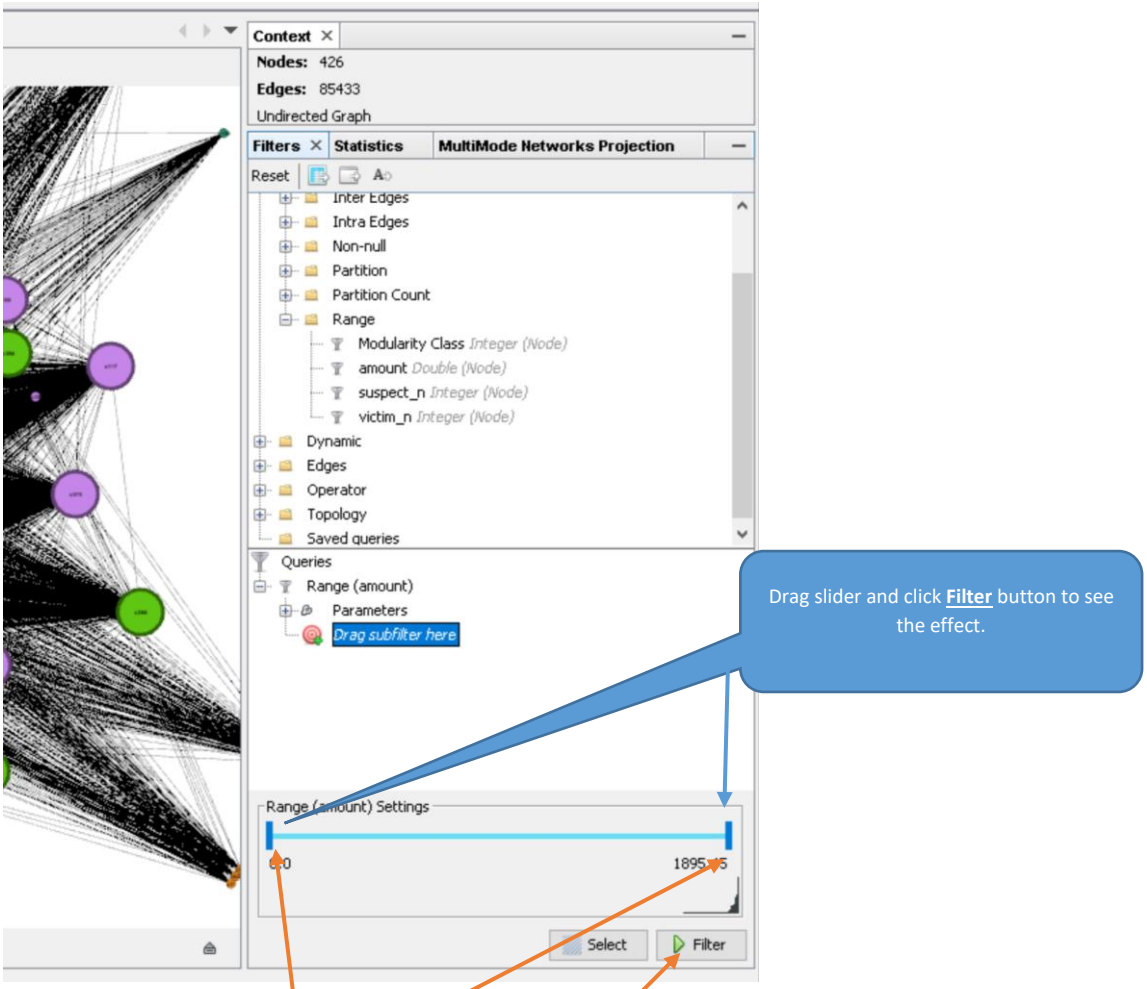


Figure 41: Drag the left slider or the right-slider and click Filter button.

We will add two more filters. First,

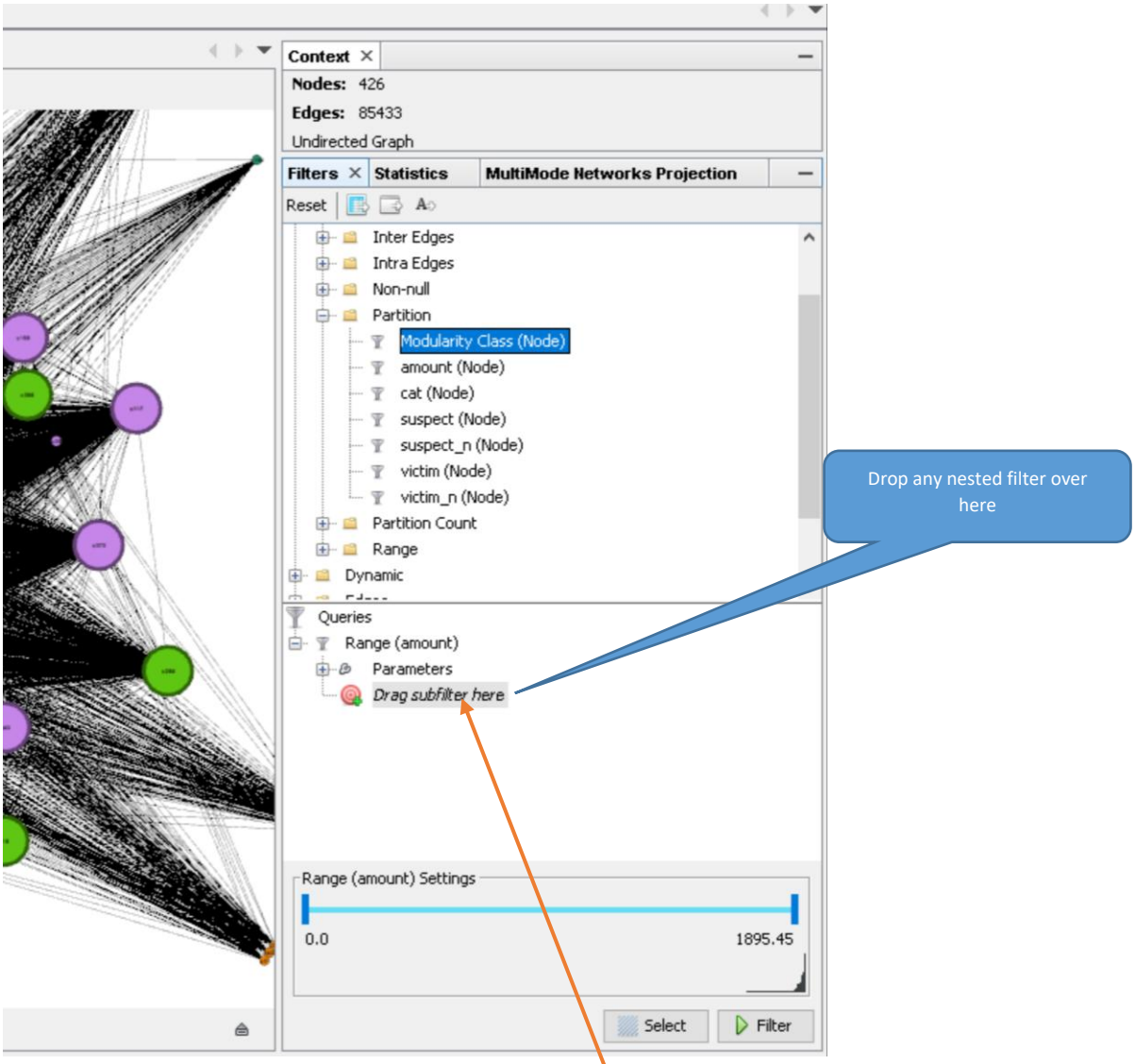


Figure 42: Click on Partition-->Modularity Class and bring it over Drag subfilter here in the lower panel.

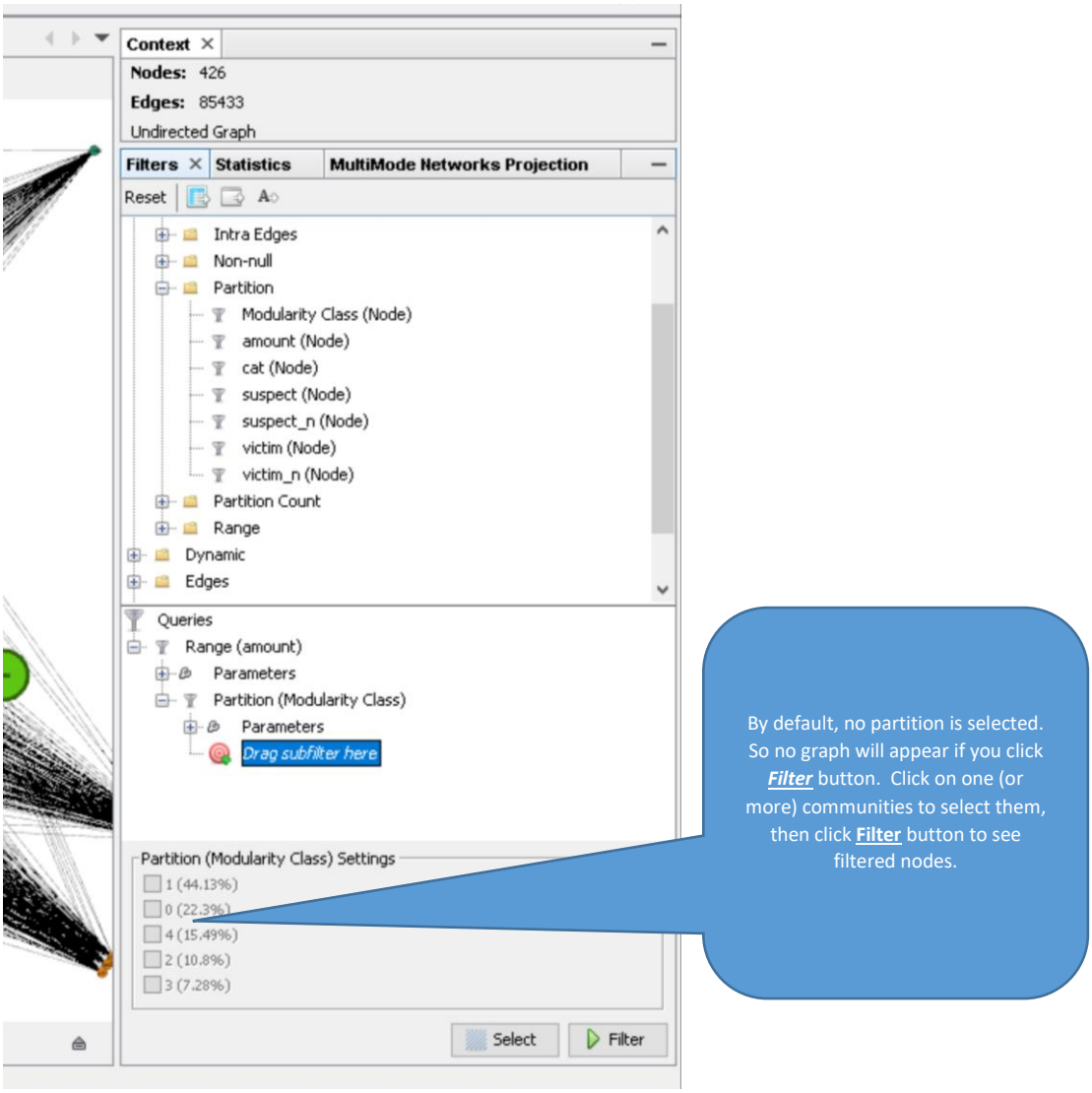


Figure 43: Two filter in place. Drag another filcter: Partition-->suspect_n over Drag subfilter here.

We will apply a third filter, 'suspect_n' to consider only those who have committed fraud.

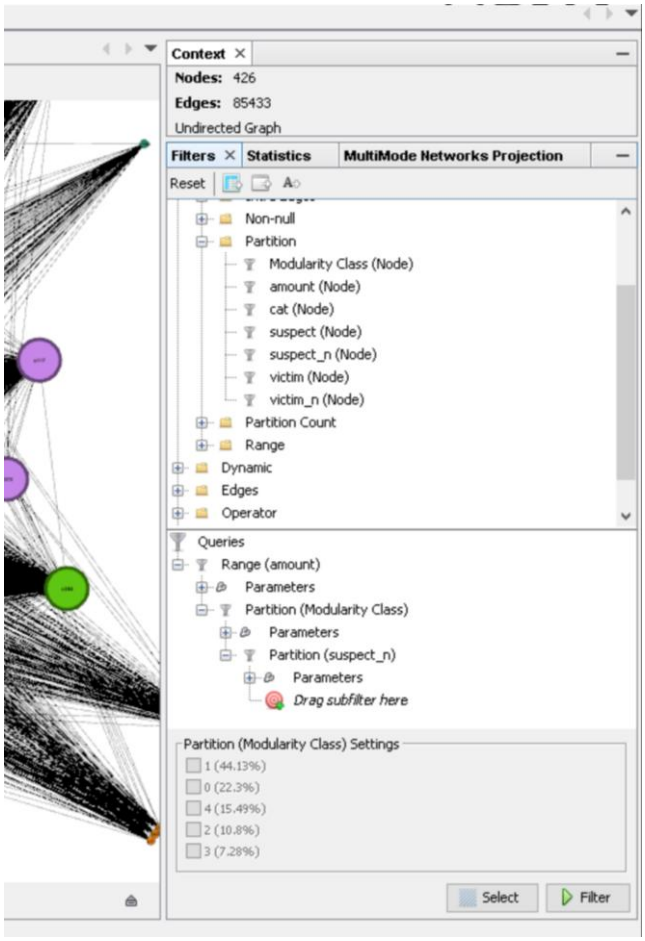


Figure 44: All three filter in place. Let us now begin to apply them.

Our objective is to see if any relationship exists between those who have committed frauds. *We will observe this relationship within each community, by turn*. And if relationship exists, how strong is the relationship.

Proceed as follows:

1. Remove clutter by setting *Amount > 20 or u_mean >20*.
2. Operate Modularity class filter,to select the *Ist community*.
3. In '*suspect_n'* filter, just select cases of 1 (frauds).

Repeat the above steps for all communities, one-by-one. Here are step wise screen shots:

If you click ***Filter*** button (without setting or selecting anything) now, nothing will appear. By default all partitions are unselected and nothing will appear when ***Filter*** is applied ie when Filter button is clicked. Select few or all Partitions (see below):

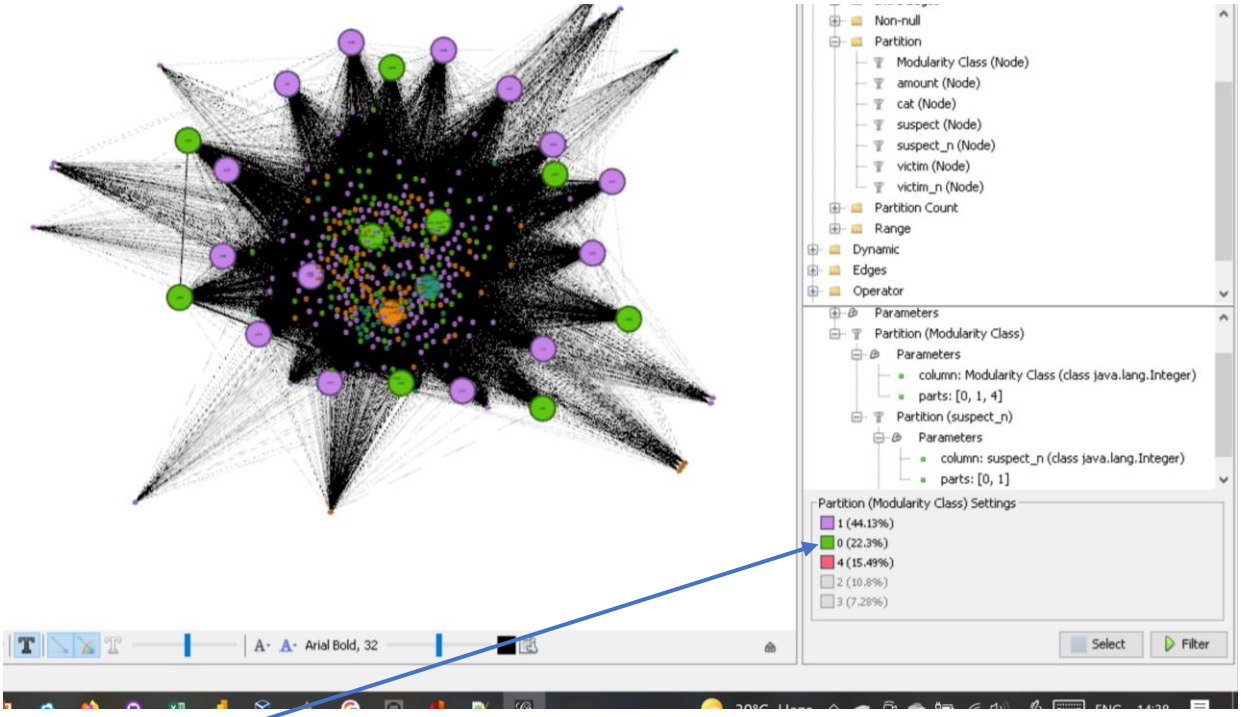


Figure 45: Three communities selected out of five.

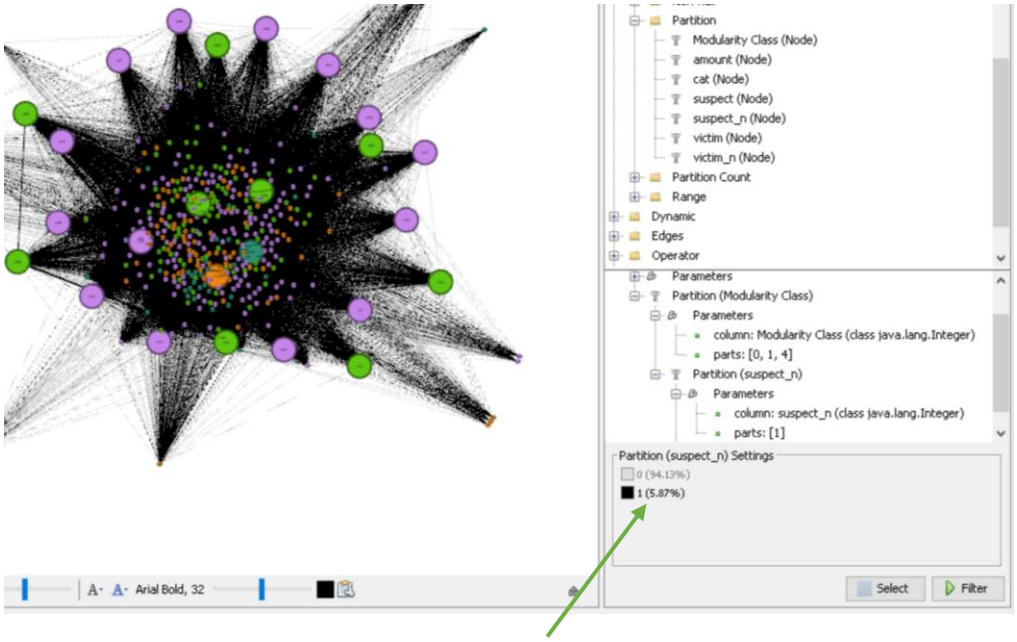


Figure 46: suspect-n, only '1' selected and not '0'.

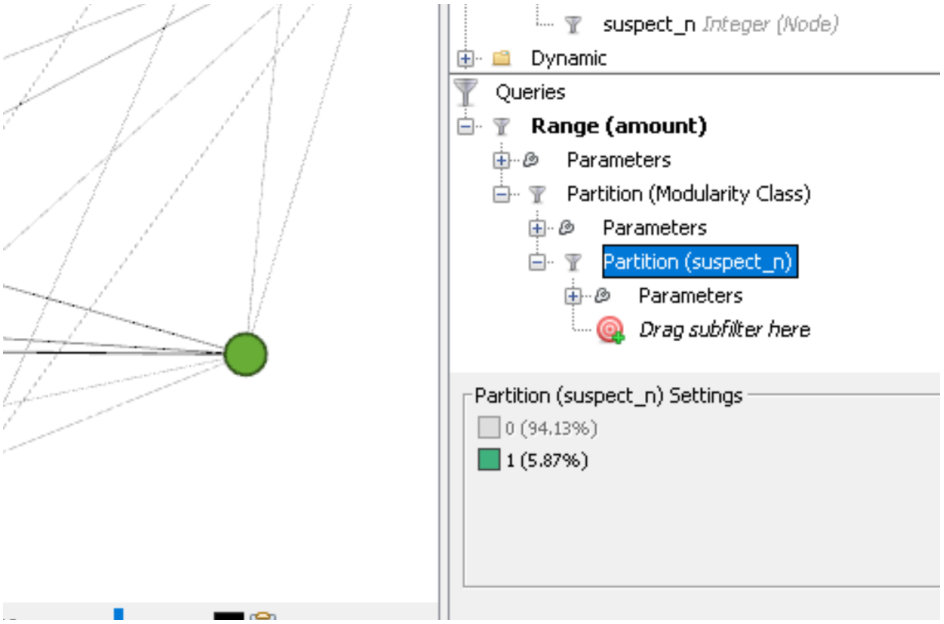


Figure 47: Zoomed picture of above. Only '1' suspect_n selected and not '0'.

In the following figure, one community is selected out of four, *Amount > 20* (or u_mean > 20) and *suspect_n* selected only for '1'**. Drag edges slider to right to darken edges and to see relationships between nodes**. Three nodes out of eight have got strong connections. This relationship needs to be examined.

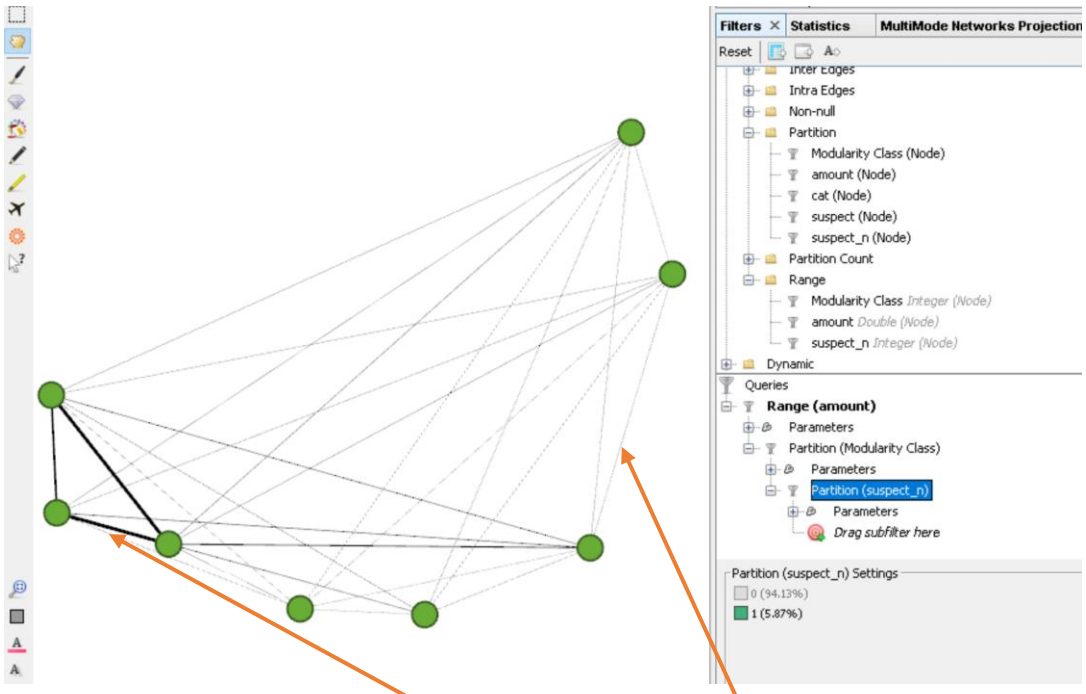
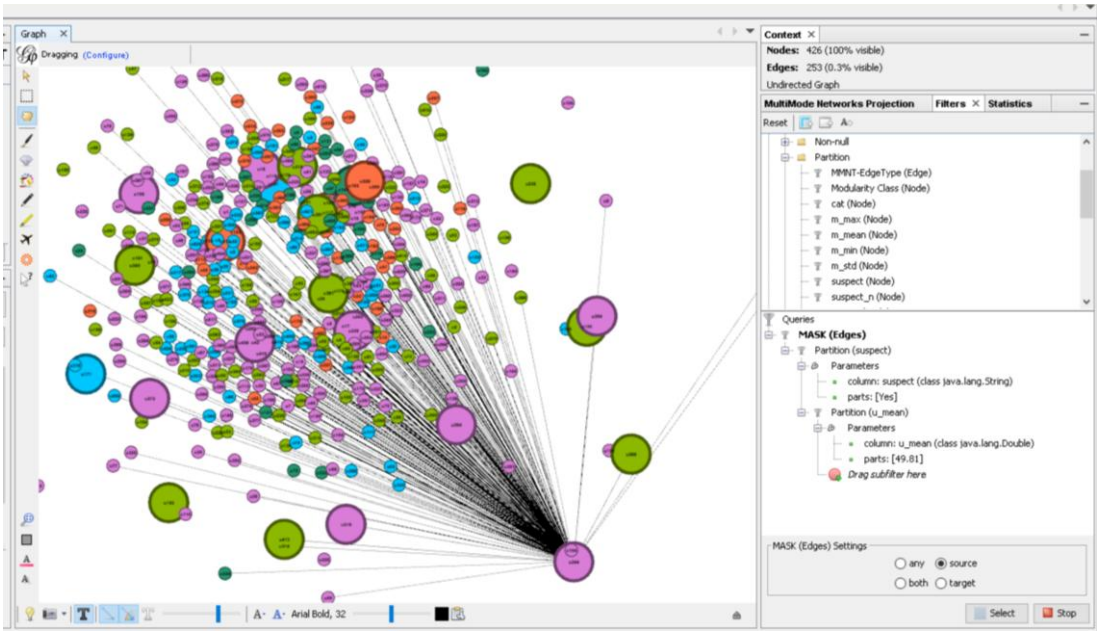


Figure 48: Three out of eight nodes have strong connections. Relatively weak relationship. Strong links needs investigations.

Masking operator

Masking operator can mask all edges but the ones you want. See [this link].



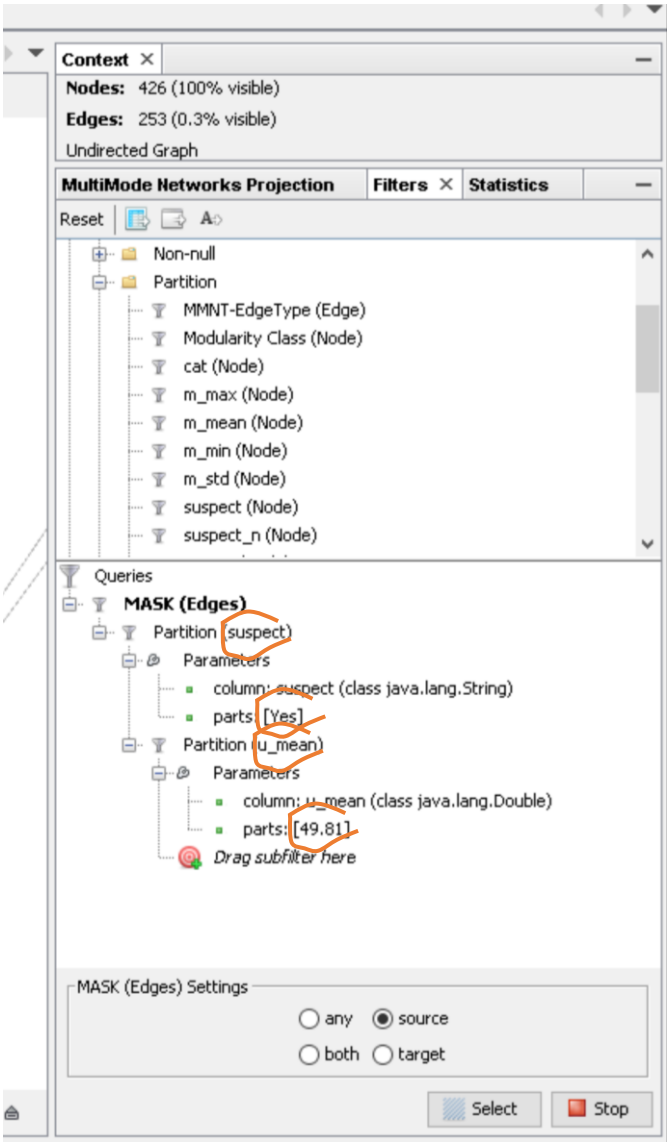
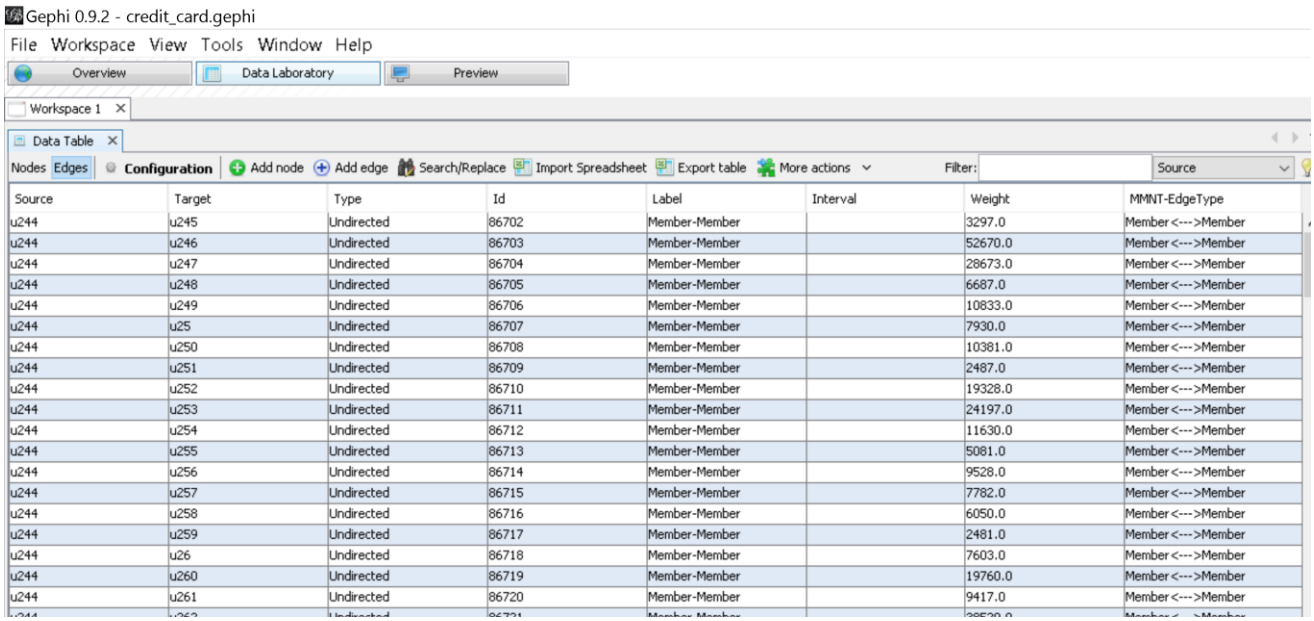


Figure 49: We have three filters here, one after another. Note that as u_mean is continuous, gephi has binned it to create distinct partitions.

The above filter-configuration says:

a) Mask all 'sources'but,
   a. where nodes where 'suspect' = 'Yes',
   b. and where u_mean partition of 49.81

Consequently, its view in *Data Laboratory* is:

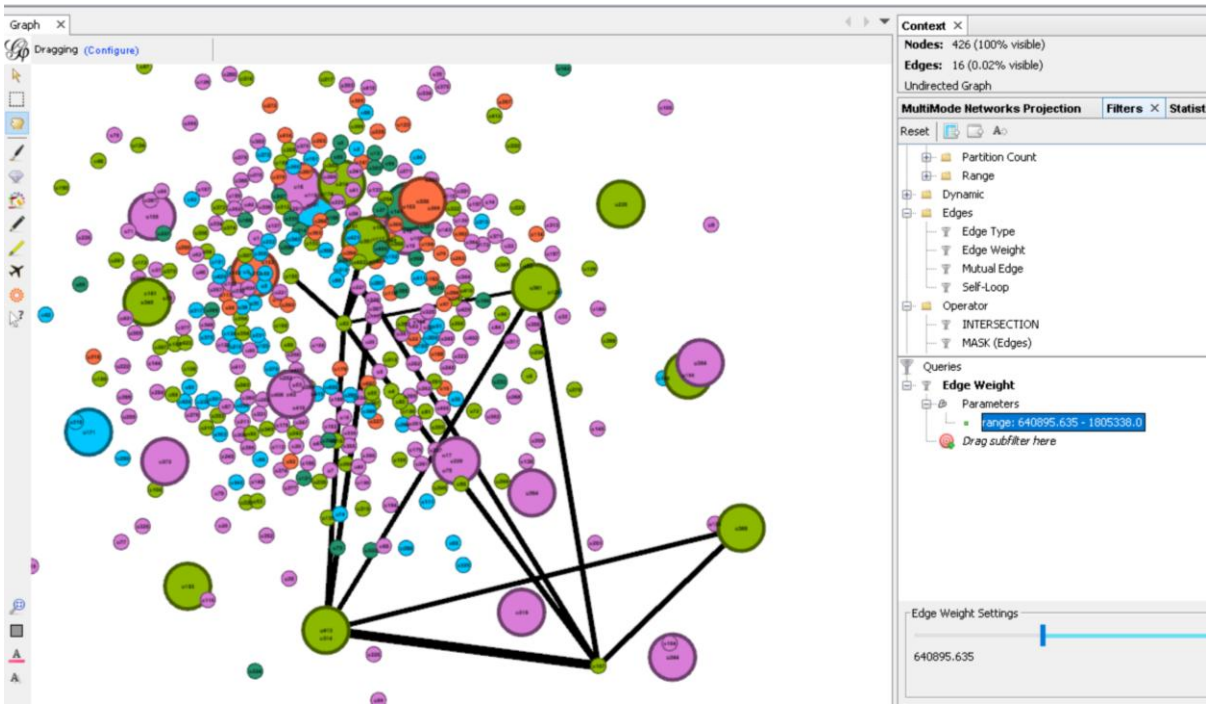


Source	Target	Type	Id	Label	Interval	Weight	MMNT-EdgeType
u244	u245	Undirected	86702	Member-Member		3297.0	Member<--->Member
u244	u246	Undirected	86703	Member-Member		52670.0	Member<--->Member
u244	u247	Undirected	86704	Member-Member		28673.0	Member<--->Member
u244	u248	Undirected	86705	Member-Member		6687.0	Member<--->Member
u244	u249	Undirected	86706	Member-Member		10833.0	Member<--->Member
u244	u25	Undirected	86707	Member-Member		7930.0	Member<--->Member
u244	u250	Undirected	86708	Member-Member		10381.0	Member<--->Member
u244	u251	Undirected	86709	Member-Member		2487.0	Member<--->Member
u244	u252	Undirected	86710	Member-Member		19328.0	Member<--->Member
u244	u253	Undirected	86711	Member-Member		24197.0	Member<--->Member
u244	u254	Undirected	86712	Member-Member		11630.0	Member<--->Member
u244	u255	Undirected	86713	Member-Member		5081.0	Member<--->Member
u244	u256	Undirected	86714	Member-Member		9528.0	Member<--->Member
u244	u257	Undirected	86715	Member-Member		7782.0	Member<--->Member
u244	u258	Undirected	86716	Member-Member		6050.0	Member<--->Member
u244	u259	Undirected	86717	Member-Member		2481.0	Member<--->Member
u244	u26	Undirected	86718	Member-Member		7603.0	Member<--->Member
u244	u260	Undirected	86719	Member-Member		19760.0	Member<--->Member
u244	u261	Undirected	86720	Member-Member		9417.0	Member<--->Member

Figure 50: Any node can be target. But source node is only suspect: Yes and partition: 49.81

Edge Weight filter:

Edge weight filter can also display strong bindings. In the following figure, just edge weight filter is applied. And one can see some strong connections.



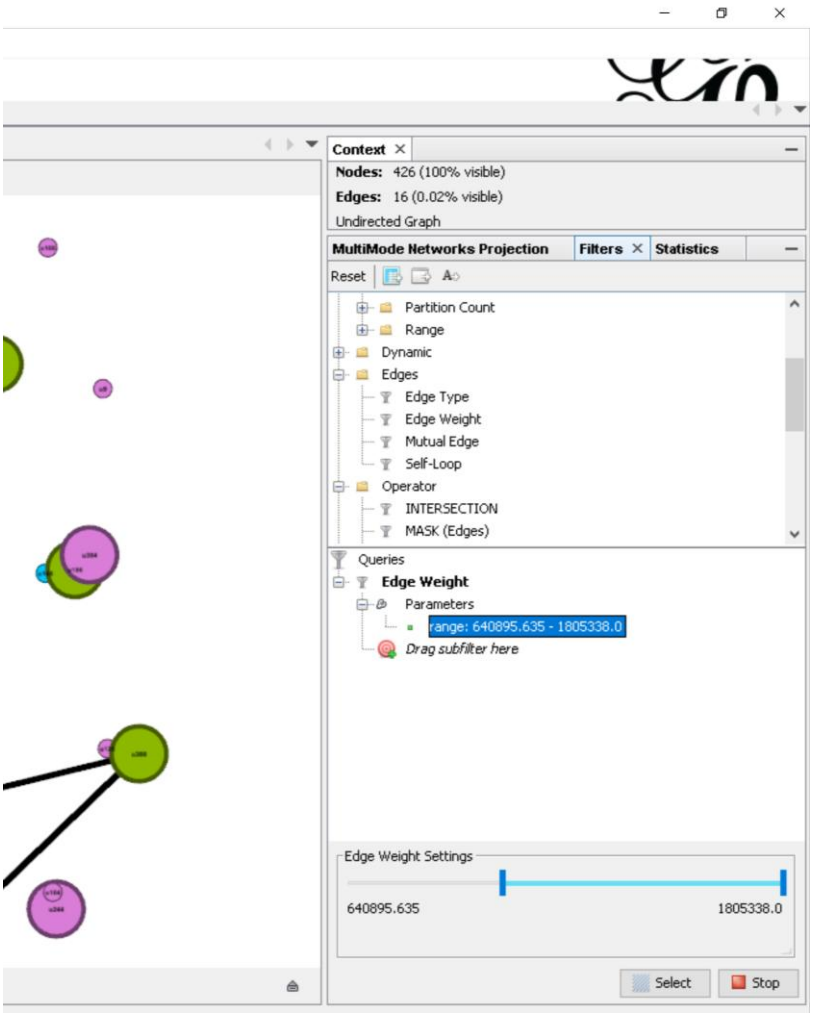


Figure 51: Edges weight filter configuration

Interpretation of Filtered Graph

Prompt:

We are using Gephi. We have Users and Merchants transaction graph.

Consider two use cases. In the first use case we have a graph of merchants and users. An edge between them denotes transaction. In this first case, irrespective of the number of interactions between a User and a Merchant, all edge weights have weight of 1.

We then apply bimodal projection on the graph to eliminate Merchant-nodes. The resulting graph has only connected users. We now calculate Modularity. We get a maximum modularity score of 1.73 when the number of structures or classes are 4.

Next, we filter the graph by 'Amount' attribute of User Node. And then by a User attribute 'suspect_n' which is 1 if the User ever committed fraud. We filter out nodes with 'suspect_n' value of 0. Lastly we mask out edges with lower weights.

In the graph we are now left with certain connected users with high edge weights. What is the importance of this observation?

Reply:

First, a flag on the number: Gephi's modularity is bounded above by 1 (theoretically it ranges from -0.5 to 1). A score of 1.73 isn't possible for standard modularity, so check how it was computed. Common culprits are the weighted/resolution settings (a resolution well above 1 can inflate values in some implementations), a different tool's output, or a misread of the report. I'd verify this before relying on the 4-community result. The rest of the answer holds either way.

What the projection edge weights mean

After a bimodal (one-mode) projection onto Users, the edge weight between two users is typically the number of merchants they share. Since the original User-Merchant edges were all weight 1, this weight is a clean measure of overlap: it doesn't reflect how often someone transacted, only how many distinct merchants two users have in common.

What your filtering leaves you with

You kept users with a high Amount, kept only suspect_n = 1 (known fraudsters), and kept only heavy edges. So the remaining structure is: known fraudulent users, with significant transaction volume, who share many of the same merchants.

Why this matters

1. **Evidence of coordinated or organized fraud.** Independent fraudsters would have little reason to overlap heavily. Many shared merchants among confirmed fraudsters suggests a ring, shared playbook, shared compromised credentials, or the same person operating multiple accounts.
2. **Identification of compromised or colluding merchants.** The shared merchants are the common thread. They could be points of compromise (card skimming, data breach), complicit merchants, or merchants with weak fraud controls that fraudsters favor for cashing out or testing cards.
3. **Guilt-by-association for detection.** Non-flagged users who are strongly connected to this cluster become high-priority suspects. You can score unlabeled users by their tie strength to confirmed fraudsters, which is a strong feature for a model or a manual review queue.
4. **Prioritization by impact.** Filtering on Amount means you're looking at the highest-loss subgraph, so investigation effort goes where the financial exposure is greatest.
5. **Link to the community structure.** Check whether these users sit in one of your modularity classes. If they concentrate in one community, that community is a fraud-prone segment worth monitoring wholesale. If they span several, it points to a cross-community network that modularity alone would miss.

Caveats

- Shared merchants can also reflect popularity: users who all shop at a huge merchant (a large marketplace, a fuel chain) will overlap by chance. Down-weight common merchants (e.g., IDF-style weighting or Jaccard/cosine normalization) so the signal isn't just popular merchants.
- Filtering on suspect_n = 1 introduces selection bias. You only see detected fraud, so the cluster may reflect where detection was strongest, not where fraud is.
- Threshold choice for "high weight" is arbitrary. Test sensitivity by varying it and seeing whether the cluster is stable.
- Correlation isn't proof of collusion. Treat it as a lead for investigation, not a conclusion.

Next steps

Compare the weight distribution of fraud-fraud edges against a baseline of random user pairs and fraud-nonfraud pairs. If fraud-fraud overlap is significantly higher, you've quantified the effect rather than just observing it. You can also restore the merchant nodes for this subset to identify which specific merchants tie the cluster together.

Prompt:

Sorry it was 0.173 not 1.73

Reply

That makes sense, and 0.173 is a valid value. It also changes how I'd read the community structure, so here's the update.

What 0.173 means

Modularity measures how much denser the connections are within communities than you'd expect by chance. Values above roughly 0.3 are usually taken to indicate meaningful community structure, and strong structure often lands at 0.5 or higher. At 0.173 with 4 classes, the user projection has only weak community structure. Users aren't cleanly separating into distinct groups based on shared merchants.

This is plausible for a transaction graph. Many users shop at the same popular merchants, so the projection is dense and overlapping, which blurs community boundaries.

How this strengthens the earlier conclusion

Your filtered subgraph (high-amount, confirmed fraudsters, heavy edges) now stands out more, not less. If the overall network has weak community structure, then a tight cluster of fraudsters with unusually high mutual overlap isn't something the general topology explains. It's an anomaly against a diffuse background. That raises the likelihood it reflects real coordination rather than an artifact of how everyone clusters around the same merchants.

What it also implies

1. **Modularity alone won't find the fraud ring.** With a score this low, the 4 communities are not a reliable way to isolate fraudsters. The filtering approach you used (attribute plus edge weight) is the better tool here.
2. **Don't treat the 4 communities as segments.** They're a weak partition, so check how fraudsters distribute across them rather than assuming each community is meaningful.
3. **The unweighted choice may be limiting.** With all edge weights at 1 in the bipartite graph, the projection captures only breadth of shared merchants. Since the overall structure is weak, it's worth testing a version weighted by transaction count or amount, or one with popular-merchant down-weighting, to see whether community structure sharpens and whether the fraud cluster stays intact.

4. **Resolution matters.** Gephi's default resolution of 1.0 may not suit this graph. Try varying it and see whether modularity and the number of communities change meaningfully. If 4 classes at 0.173 is stable across settings, that's a reliable finding that the graph is weakly clustered.

   Suggested check

   Compute the fraction of confirmed fraudsters in each of the 4 communities versus the overall fraud rate. If fraudsters are concentrated in one community despite the low modularity, that community is a useful risk segment. If they're spread evenly, the heavy-edge fraud cluster is the real signal, and it's cutting across the general community structure.

####### Done ############